Dynamics CRM plugins

In Dynamics 365, a **plugin** is a custom piece of code that developers create to extend and customize the functionality of the CRM platform. It acts as a response mechanism to specific events or operations that take place within the Dynamics 365 system. These events can include actions like creating, updating, or deleting records, as well as more specialized events like assigning records or retrieving data. Plugins encapsulate custom business logic written by developers. This logic dictates what actions the plugin should take when a particular event occurs. This could involve manipulating data, performing additional calculations, or enforcing specific business rules. The purpose is to tailor the behavior of the CRM system to meet the unique needs of an organization. Before a plugin can execute, it needs to be registered with the Dynamics 365 system. Registration involves specifying details such as the event, stage, and entity to which the plugin is bound. The registration process is typically managed using tools like the Plugin Registration Tool.

There are different types of plugins based on when they execute in the **event execution pipeline**.

To describe the various stages, we first need to explain the concept of database transaction.

A **database transaction** symbolizes a unit of work, performed within a database management system (or similar system) against a database, that is treated in a coherent and reliable way independent of other transactions. A transaction generally represents any change in a database. Transactions in a database environment have two main purposes:

1. To provide reliable units of work that allow correct recovery from failures and keep a database consistent even in cases of system failure. For example: when execution prematurely and unexpectedly stops (completely or partially) in which case many operations upon a database remain uncompleted, with unclear status.

2. To provide isolation between programs accessing a database concurrently. If this isolation is not provided, the programs' outcomes are possibly erroneous.

In a database management system, a transaction is a single unit of logic or work, sometimes made up of multiple operations. Any logical calculation done in a consistent mode in a database is known as a transaction. One example is a transfer from one bank account to another: the complete transaction requires subtracting the amount to be transferred from one account and adding that same amount to the other. A database transaction, by definition, must be *atomic* (it must either be complete in its entirety or have no effect whatsoever), *consistent* (it must conform to existing constraints in the database), *isolated* (it must not affect other transactions) and *durable* (it must get written to persistent storage).

Pre-validation Stage: developers have the opportunity to perform additional checks on data integrity, enforce specific business rules, or make adjustments to the data based on specific requirements. The primary purpose is to enable custom validation or manipulation of data before it undergoes the platform's built-in validation. When it is said that the pre-validation stage occurs outside the database transaction in Dynamics 365, it means that the checks and adjustments made during this stage do not permanently affect the database. Any changes made here are temporary and do not impact the actual state of the database. Plugins registered in the pre-validation stage have the ability to cancel the entire operation if necessary. If custom validation logic detects issues with the data that would violate business rules, the plugin can prevent the invalid data from progressing further in the pipeline.

Pre-operation Stage: developers use the pre-operation stage to manipulate data or perform additional validation before it undergoes the primary operation. This can include adjusting attribute values, enforcing business rules, or ensuring data consistency. Unlike the pre-validation stage, the actions taken during the pre-operation stage are within the context of the database transaction. Changes made here can potentially affect the outcome of the main operation, but they are still subject to the ACID properties (Atomicity, Consistency, Isolation, Durability). Changes made by pre-operation plugins are part of the transaction, and they contribute to the overall transactional behavior of the system.

Core Operation Stage: the core operation is the main action that the system performs during an event. It happens right after the pre-operation stage in the event execution pipeline. This operation is specific to the type of event triggered, such as creating, updating, deleting, or retrieving records. Think of it as the heart of the event. If the event is about creating a record, the core operation is inserting that record into the database. If it is an update event, the core operation involves modifying the attributes of an existing record. For a delete event, it means removing a record from the database. This core operation is part of the overall database transaction, ensuring that any changes made are handled with the principles of ACID. Plugins cannot be registered at this specific stage because the core operation stage is crucial for maintaining the integrity of the system's behavior, and introducing new plugins at this stage could disrupt the predefined workflow and potentially conflict with the expected actions of the core operation.

Post Operation Stage: it occurs within the same transaction context as the core operation, allowing developers to perform additional actions or validations based on the outcome of the main operation. The post-operation stage provides an opportunity for developers to respond to the results of the core operation. For example, after a record is created, updated, or deleted, post-operation plugins can be used to trigger further processes, update related records, or perform additional calculations.

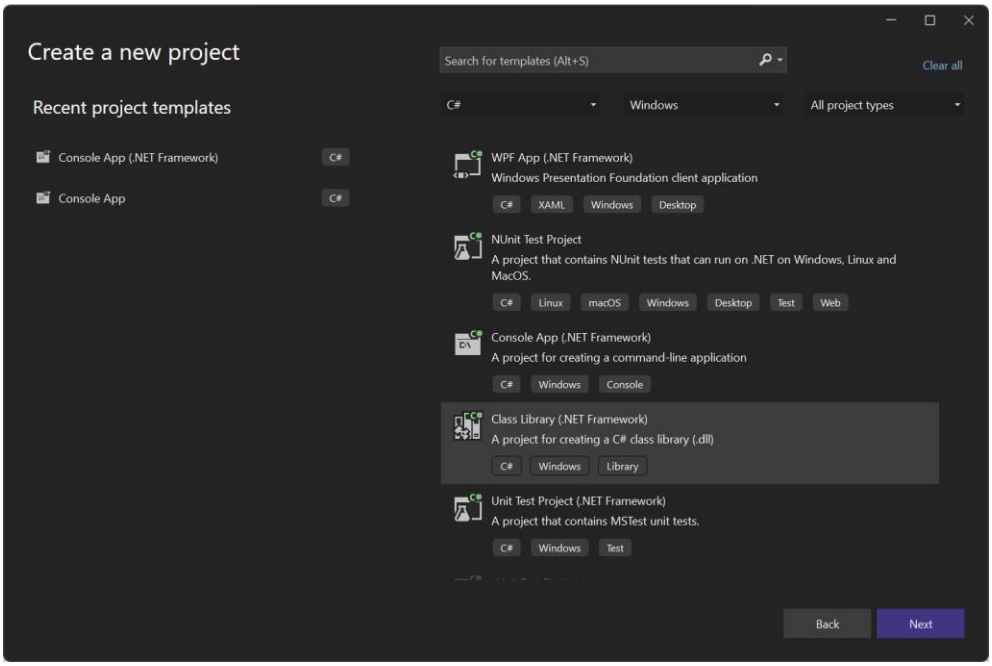
Synchronous plugins operate in real-time during the event execution pipeline, while **asynchronous plugins** execute in the background, allowing for non-blocking operations.

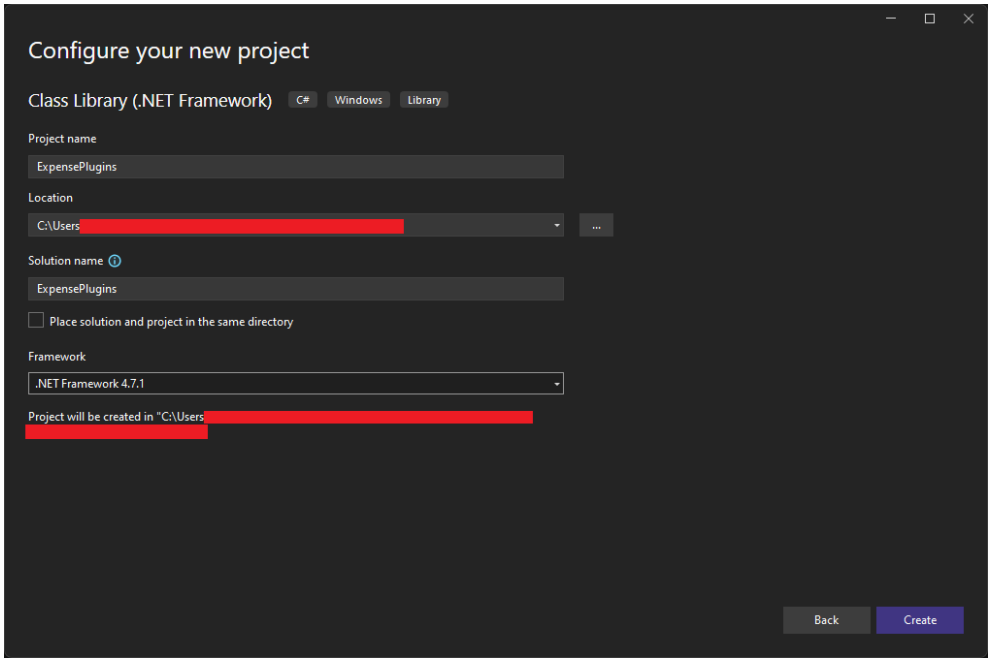
Developers use tools like Visual Studio and the Dynamics 365 SDK for plugin development. These tools provide the necessary libraries and resources for creating, debugging, and deploying plugins. Debugging plugins involves techniques like attaching to the process during development or incorporating tracing statements in the code to identify and address issues effectively.

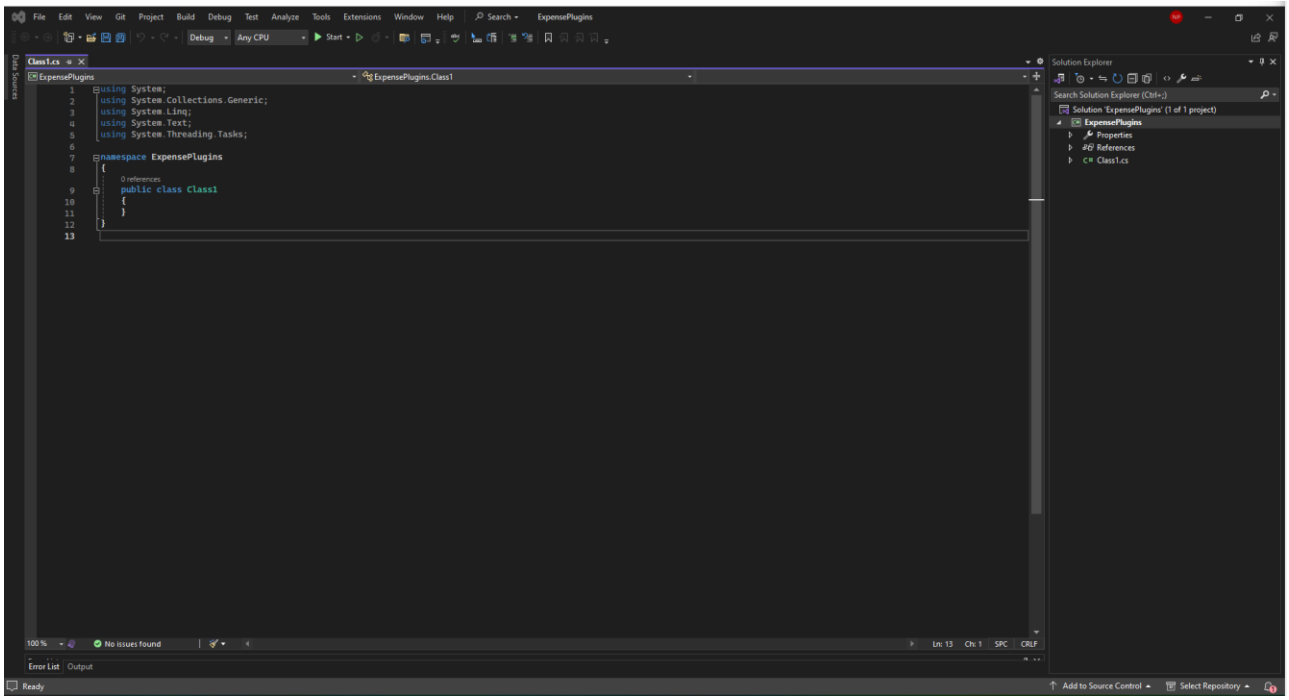
1. Writing and registering plugins

Navigate to https://learn.microsoft.com/en-us/power-apps/developer/data-platform/tutorial-write-plug-in for a detailed documentation on how to write and register a plugin.

First of all, we create in Visual Studio a new project of type Class Library (.NET Framework).







Rename the file from Class1 to PluginExample.

In writing a Dynamics 365 plugin, we use an interface called IPlugin. The IPlugin interface is defined in the Microsoft.Xrm.Sdk namespace and serves as the base interface for custom plugin classes. It includes a method named Execute, which is the method where the core logic of the plugin is implemented. This is called when the plugin is triggered in response to a specific event, such as the creation, updating, or deletion of a record. The Execute method typically takes a parameter of type IServiceProvider, which provides access to the execution context of the plugin. This context includes information about the event, the target entity, and other relevant details. Within the Execute method, we can interact with the Dynamics 365 service to retrieve or modify data. The service provider gives us access to services like IOrganizationService, which allows to perform CRUD operations on entities. The Execute method should include proper error handling to ensure that the plugin behaves predictably even in case of unexpected situations. This might involve logging errors, rolling back transactions, or taking other appropriate actions.

Consider the following example taken from the official documentation.

```
public class FollowupPlugin : IPlugin
{
    public void Execute(IServiceProvider serviceProvider)
    {
        throw new NotImplementedException();
    }
}
```

Having the Execute method is mandatory, and only the code inside this method will be executed. So, to use a method which outside of Execute, we must call it inside the body of Execute.

Let us replace the content of the Execute method with the following code:

```
// Obtain the tracing service
ITracingService tracingService =
(ITracingService)serviceProvider.GetService(typeof(ITracingService));

// Obtain the execution context from the service provider.
IPluginExecutionContext context = (IPluginExecutionContext)
    serviceProvider.GetService(typeof(IPluginExecutionContext));

// The InputParameters collection contains all the data passed in the message request.
if (context.InputParameters.Contains("Target") &&
    context.InputParameters["Target"] is Entity)
{
    // Obtain the target entity from the input parameters.
    Entity entity = (Entity)context.InputParameters["Target"];

    // Obtain the IOrganizationService instance which you will need for
    // web service calls.
    IOrganizationServiceFactory serviceFactory =
        (IOrganizationServiceFactory)serviceProvider.GetService(typeof(IOrganizationServiceFactory));
    IOrganizationService service = serviceFactory.CreateOrganizationService(context.UserId);

    try
    {
        // Plug-in business logic goes here.
    }

    catch (FaultException<OrganizationServiceFault> ex)
    {
        throw new InvalidPluginExecutionException("An error occurred in FollowUpPlugin.", ex);
    }

    catch (Exception ex)
    {
        tracingService.Trace("FollowUpPlugin: {0}", ex.ToString());
        throw;
    }
}
```

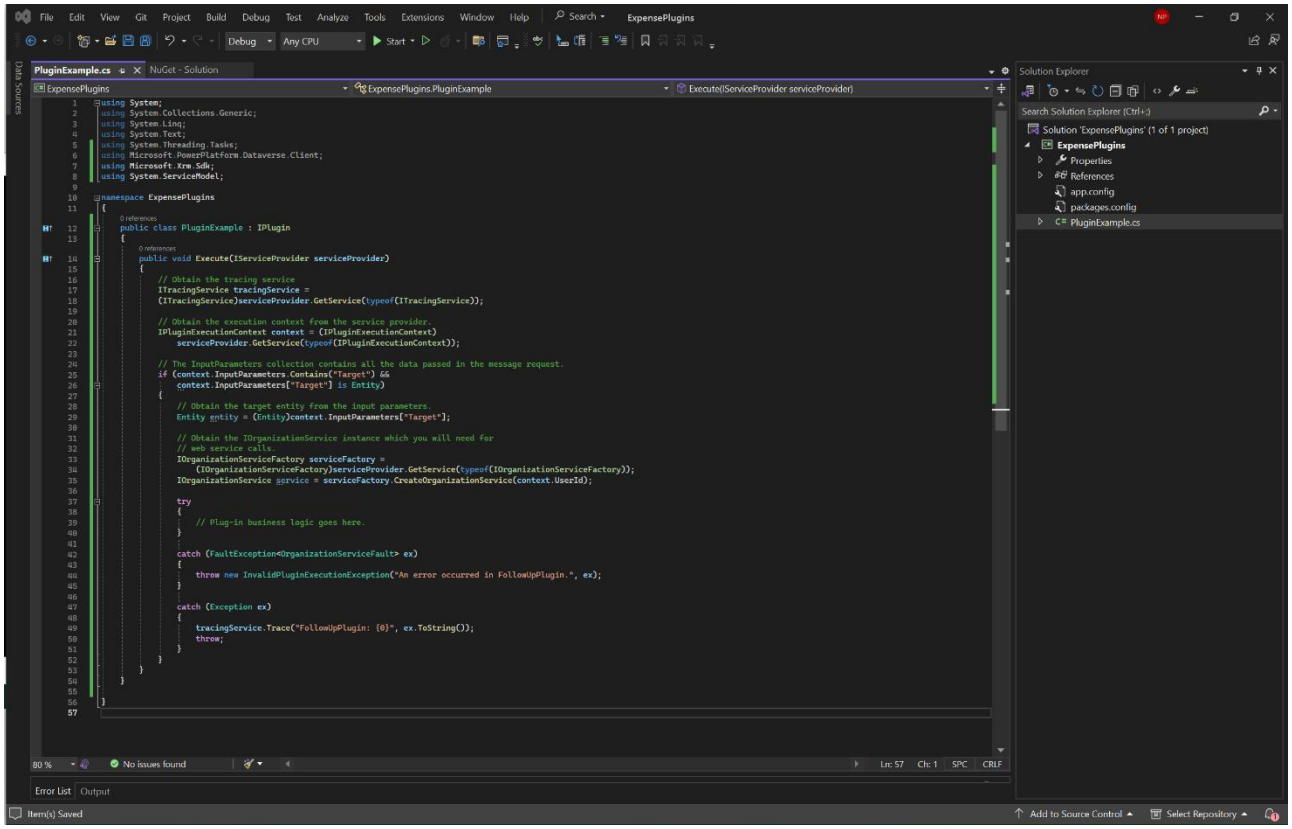
The ITracingService interface provides a way to log plugin run-time information. This method of logging information is especially useful for sandboxed plugins registered with Dataverse that cannot otherwise be debugged using a debugger. The tracing information is displayed in a dialog of the Microsoft Dynamics 365 Web application only if an exception is passed from a plugin back to the platform.

IPluginExecutionContext interface defines the contextual information passed to a plugin at run-time. It contains information that describes the run-time environment that the plugin is executing in, information related to the execution pipeline, and entity business information.

IOrganizationService interface provides programmatic access to the metadata and data for an organization.

Notice that whatever code we write it will be inside a try block, so that eventual exceptions will be properly handled. tracingService is used to catch such exceptions.

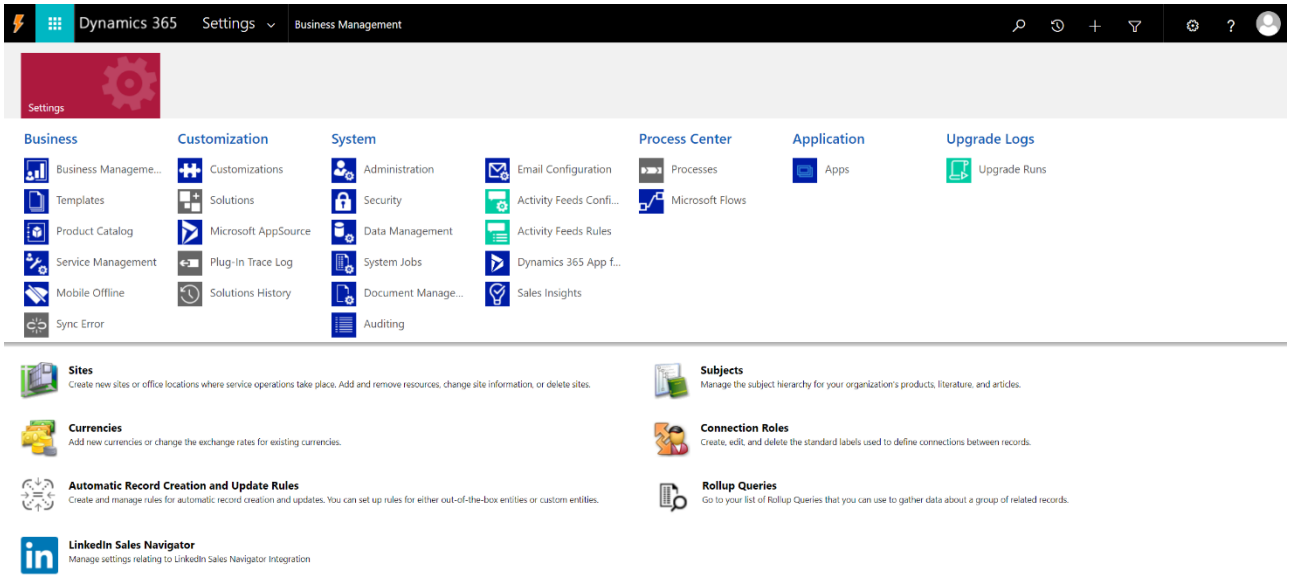
Notice the namespaces we must use to write a working plugin.



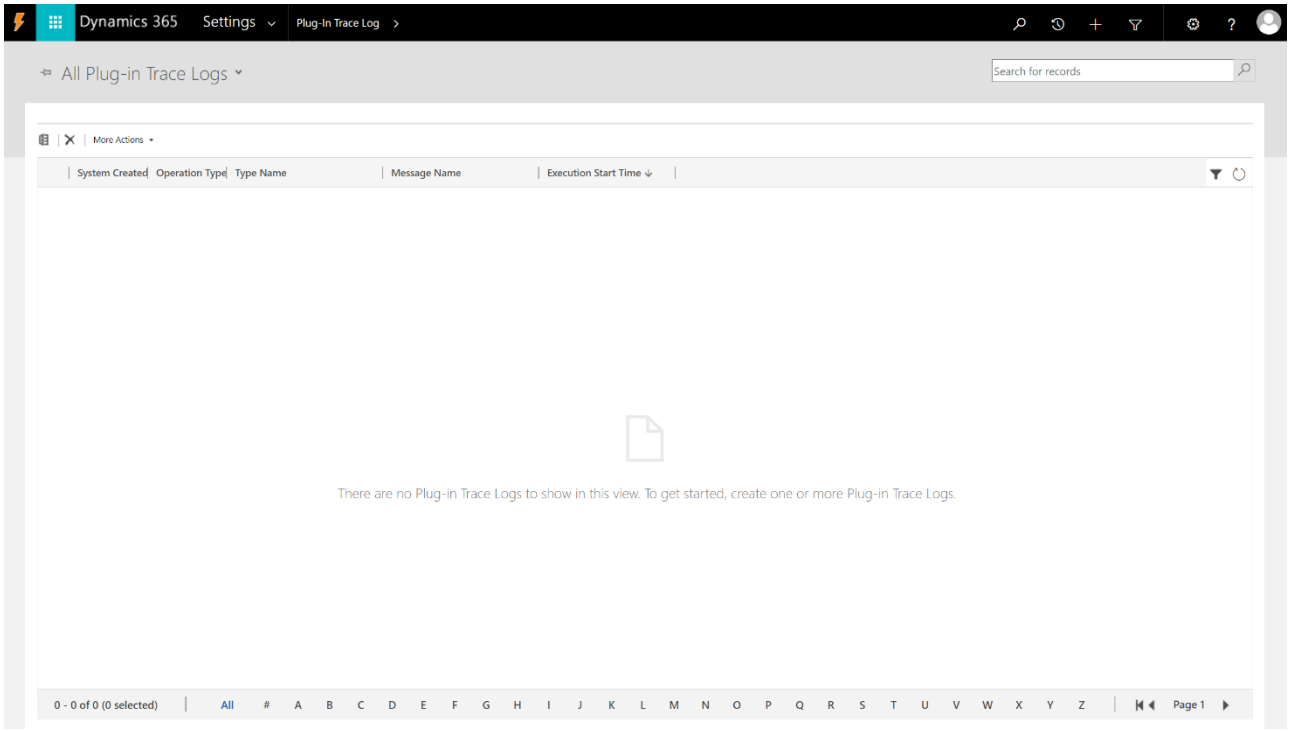
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Microsoft.PowerPlatform.Dataverse.Client;
using Microsoft.Xrm.Sdk;
using System.ServiceModel;

namespace ExpensePlugins
{
    public class PluginExample : IPlugin
    {
        public void Execute(IServiceProvider serviceProvider)
        {
            // Obtain the tracing service
            ITracingService tracingService =
            (ITracingService)serviceProvider.GetService(typeof(ITracingService));

            // Obtain the execution context from the service provider.
            IPluginExecutionContext context = (IPluginExecutionContext)
                serviceProvider.GetService(typeof(IPluginExecutionContext));

            // The InputParameters collection contains all the data passed in the message request.
            if (context.InputParameters.Contains("Target") &&
                context.InputParameters["Target"] is Entity)
            {
                // Obtain the target entity from the input parameters.
                Entity entity = (Entity)context.InputParameters["Target"];

                // Obtain the IOrganizationService instance which you will need for
                // web service calls.
                IOrganizationServiceFactory serviceFactory =
                    (IOrganizationServiceFactory)serviceProvider.GetService(typeof(IOrganizationServiceFactory));
                IOrganizationService service = serviceFactory.CreateOrganizationService(context.UserId);

                try
                {
                    // Plug-in business logic goes here.
                }

                catch (FaultException<OrganizationServiceFault> ex)
                {
                    throw new InvalidPluginExecutionException("An error occurred in FollowUpPlugin.", ex);
                }

                catch (Exception ex)
                {
                    tracingService.Trace("FollowUpPlugin: {0}", ex.ToString());
                    throw;
                }
            }
        }
    }
}
```

We already talked about the tracing service. This is not enabled by default in Dynamics 365.

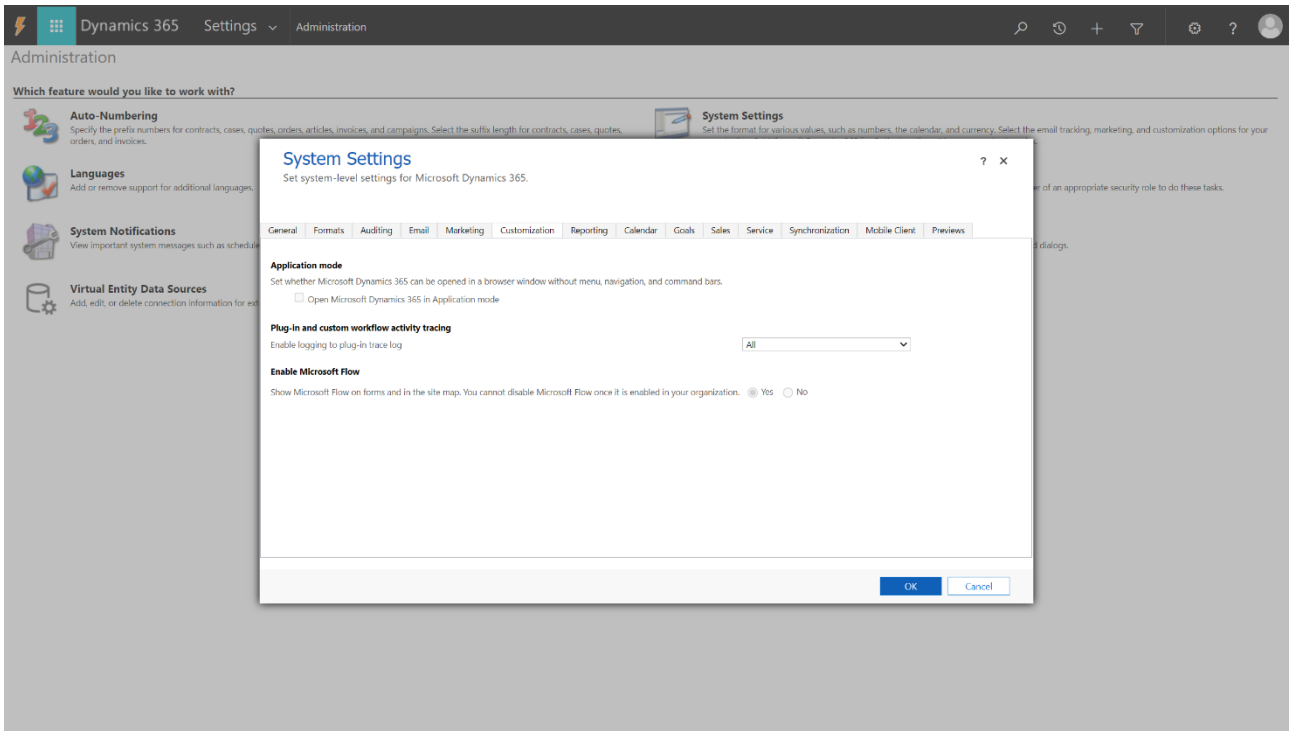
Go to Advanced Settings in Dynamics 365.



Click on Plug-In Trace Log. This is where any tracing logs we receive will be stored.



To enable the tracing service, we must go to Administration -> System Settings. Under Customization we have Plug-in and custom workflow activity tracing to be configured. In our case we set Enable logging to plug-in trace log to All.



Microsoft automatically manages the storage of these log records.

Consider the following code. The test method is outside of the Execute method. In a scenario in which we need such method to be run, we need to call it from the body of Execute.

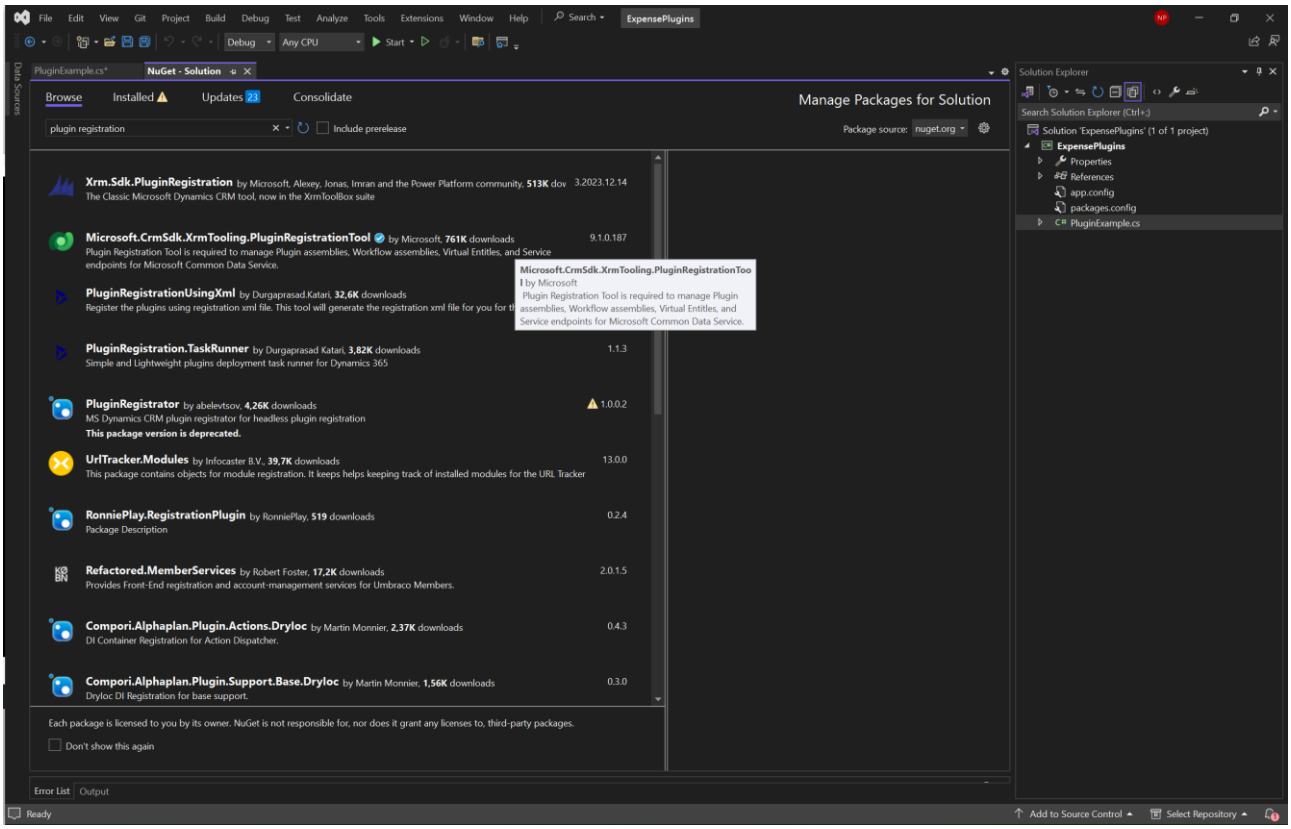
```
using Microsoft.Xrm.Sdk;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.ServiceModel;
using Microsoft.Xrm.Sdk.Query;
using Microsoft.PowerPlatform.Dataverse.Client;

namespace ExpensePlugins
{
    0 references
    public class Contact : IPlugin
    {
        0 references
        public void Execute(IServiceProvider serviceProvider)
        {
            // Obtain the tracing service
            ITracingService tracingService =
            (ITracingService)serviceProvider.GetService(typeof(ITracingService));

            // Obtain the execution context from the service provider.
            IPluginExecutionContext context = (IPluginExecutionContext)
                serviceProvider.GetService(typeof(IPluginExecutionContext));

            // The InputParameters collection contains all the data passed in the message request.
            if (context.InputParameters.Contains("Target") &&
                context.InputParameters["Target"] is Entity)
            {
                // Obtain the target entity from the input parameters.
                Entity entity = (Entity)context.InputParameters["Target"];

                // Obtain the IOrganizationService instance which you will need for
                // web service calls.
                IOrganizationServiceFactory serviceFactory =
                    (IOrganizationServiceFactory)serviceProvider.GetService(typeof(IOrganizationServiceFactory));
                IOrganizationService service = serviceFactory.CreateOrganizationService(context.UserId);

                try
                {
                    test(service);
                }

                catch (FaultException<OrganizationServiceFault> ex)
                {
                    throw new InvalidPluginExecutionException("An error occurred in FollowUpPlugin.", ex);
                }

                catch (Exception ex)
                {
                    tracingService.Trace("FollowUpPlugin: {0}", ex.ToString());
                    throw;
                }
            }
        }


        1 reference
        public static void test(IOrganizationService service)
        {
            Entity contactCreate = new Entity("contact");
            // string primary name
            contactCreate["lastname"] = "Console";
            contactCreate["firstname"] = "App";
            contactCreate["emailaddress1"] = "test@123.com";
            service.Create(contactCreate);

        }
    }
}
```

Let us modify the content of the try block as follows.

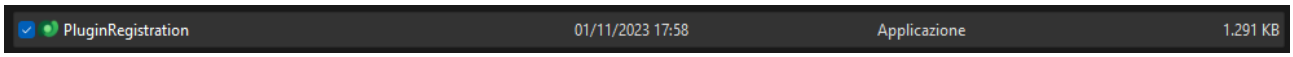
```
try
{
    if (entity.LogicalName == "contact")
    {
        string email = entity.Attributes["emailaddress1"].ToString();
        tracingService.Trace(email);
    }
}
```

If the target entity has logical name contact, the plugin retrieves the value of the emailaddress1 attribute from the entity and logs it using the tracing service.

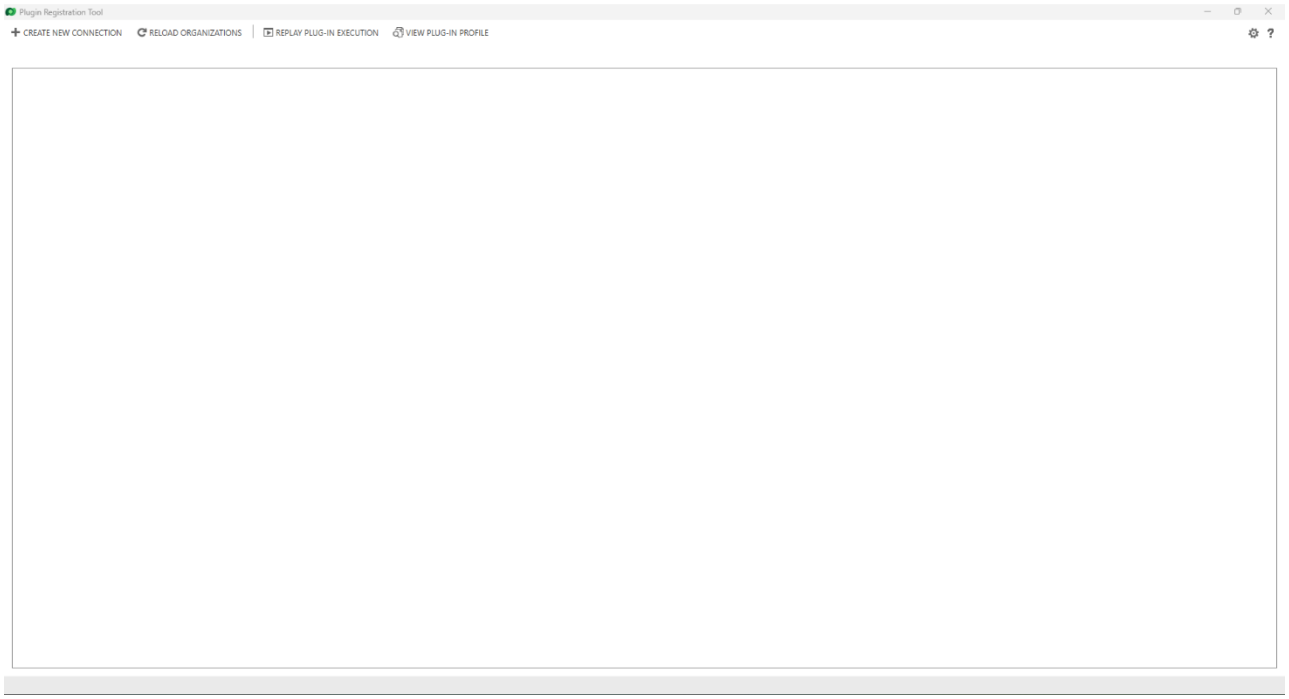
Assuming that our plugin is ready to be registered, we proceed by installing the Microsoft.CrmSdk.XrmTooling.PluginRegistrationTool package.



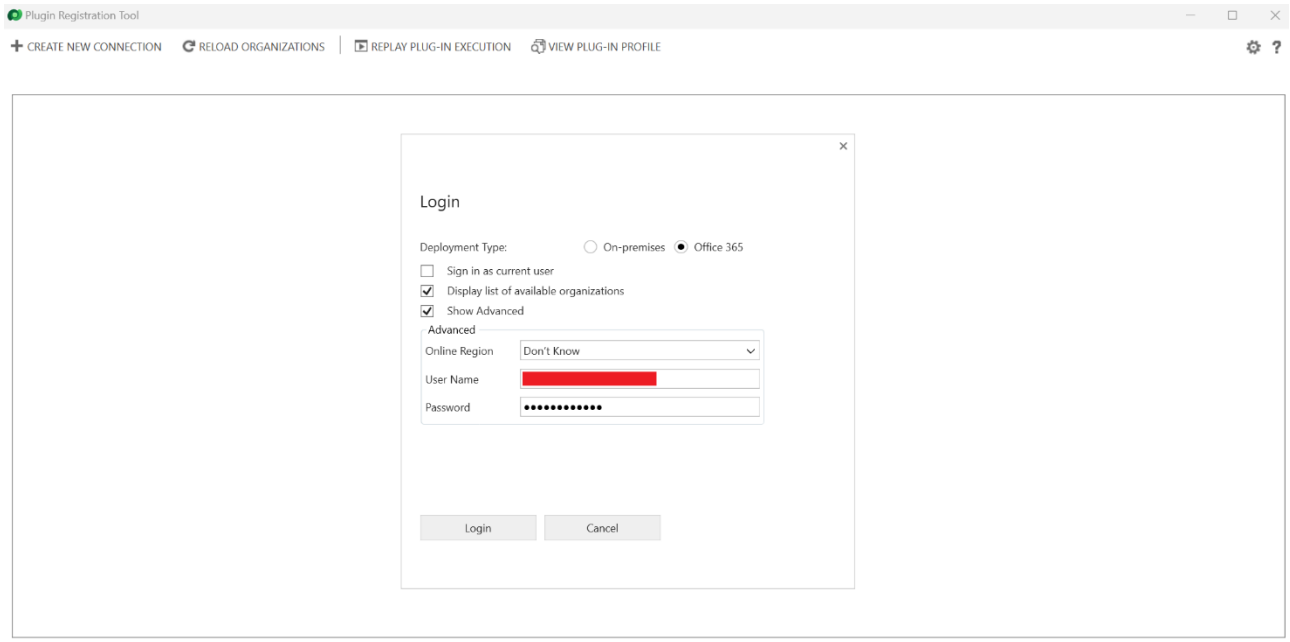
Once the download is completed, navigate to the folder containing the solution, open the PluginRegistrationTool package inside packages and double click on the PluginRegistration app contained in tools.



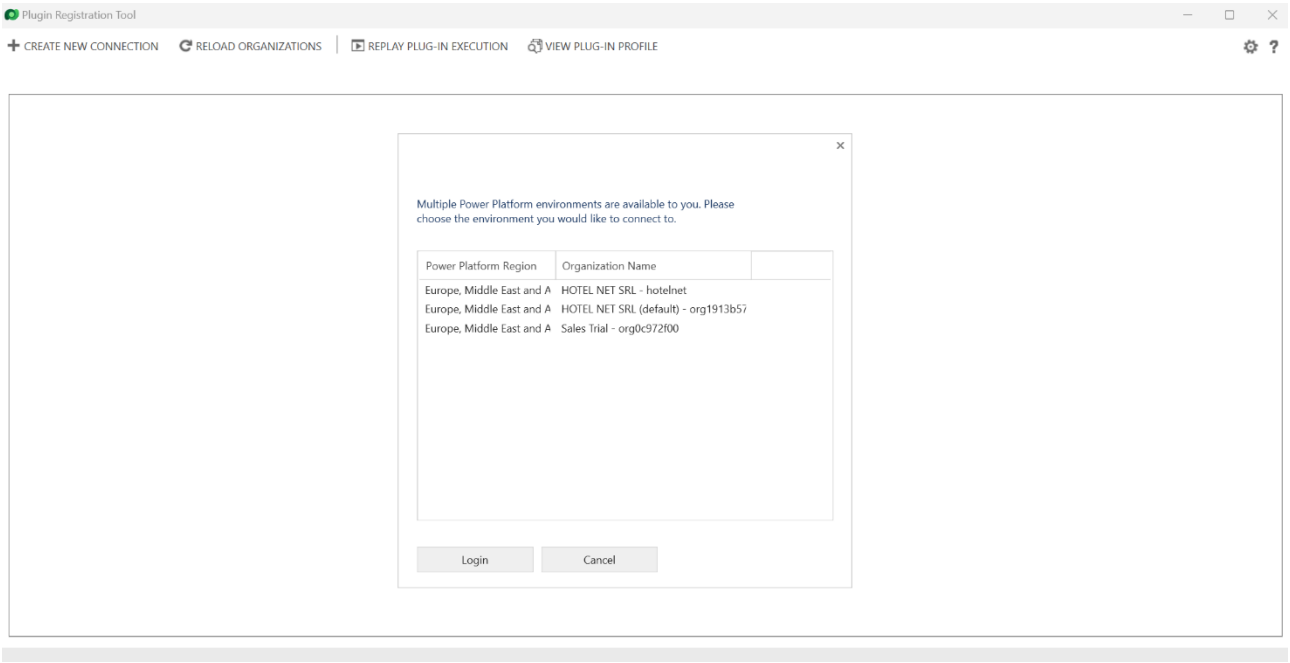
Once the application opens, it looks like the following figure.



Click on + CREATE NEW CONNECTION. Here we must enter our personal credentials.

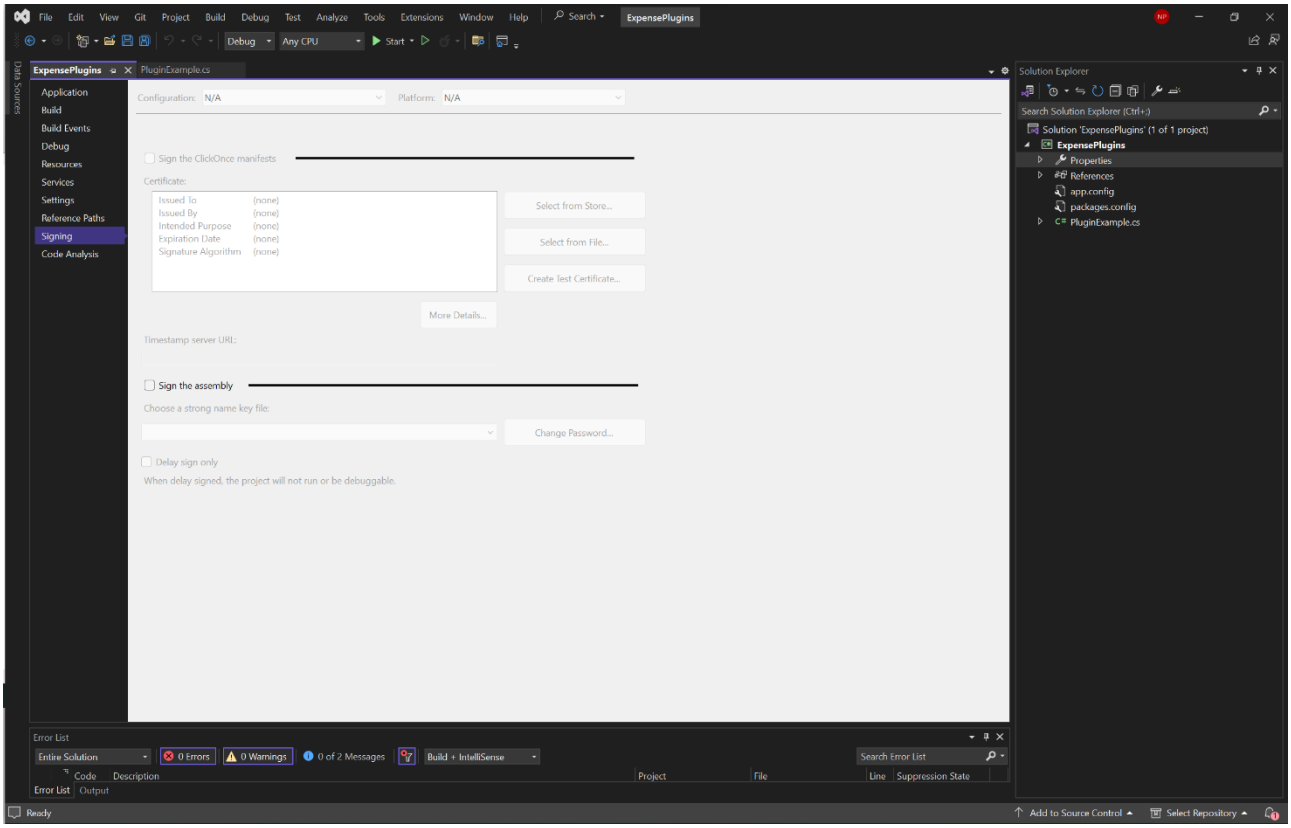


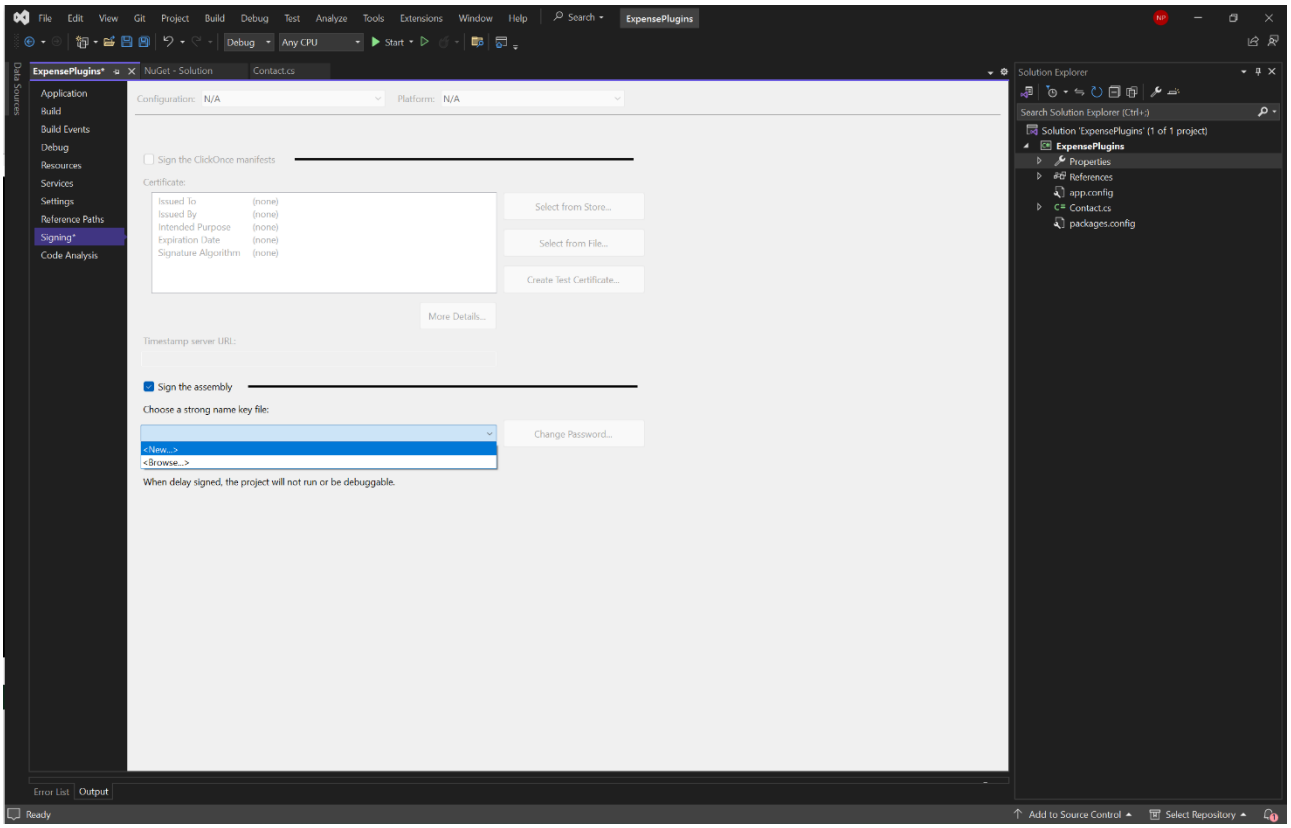
After clicking on Login, the app will show a list of the accessible environments.

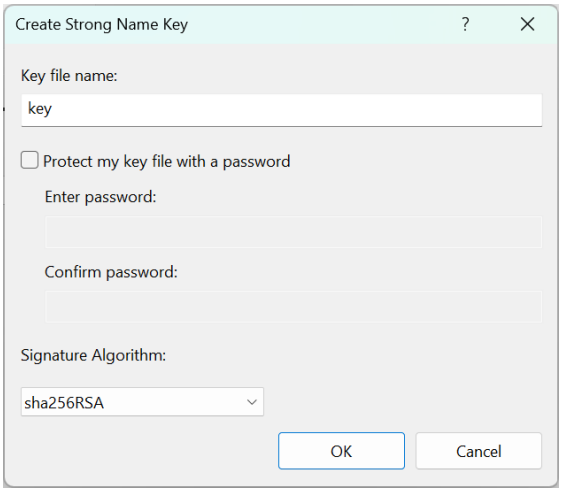


In our case we select Sales Trial.

A further step before actually registering the plugin consists in signing it. To do that, go to Visual Studio and click on Properties inside the ExpensePlugins project.



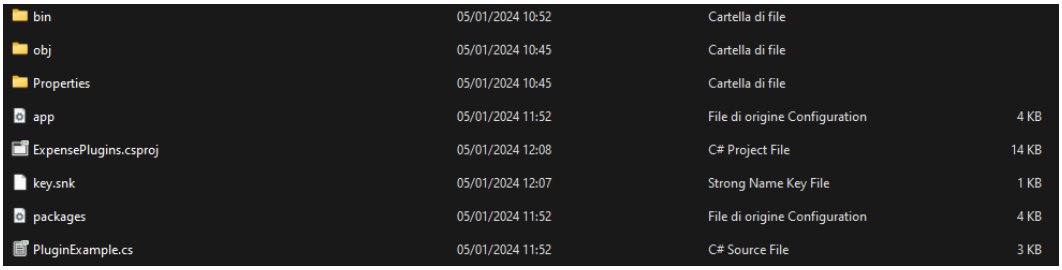




In this example we do not set a password.

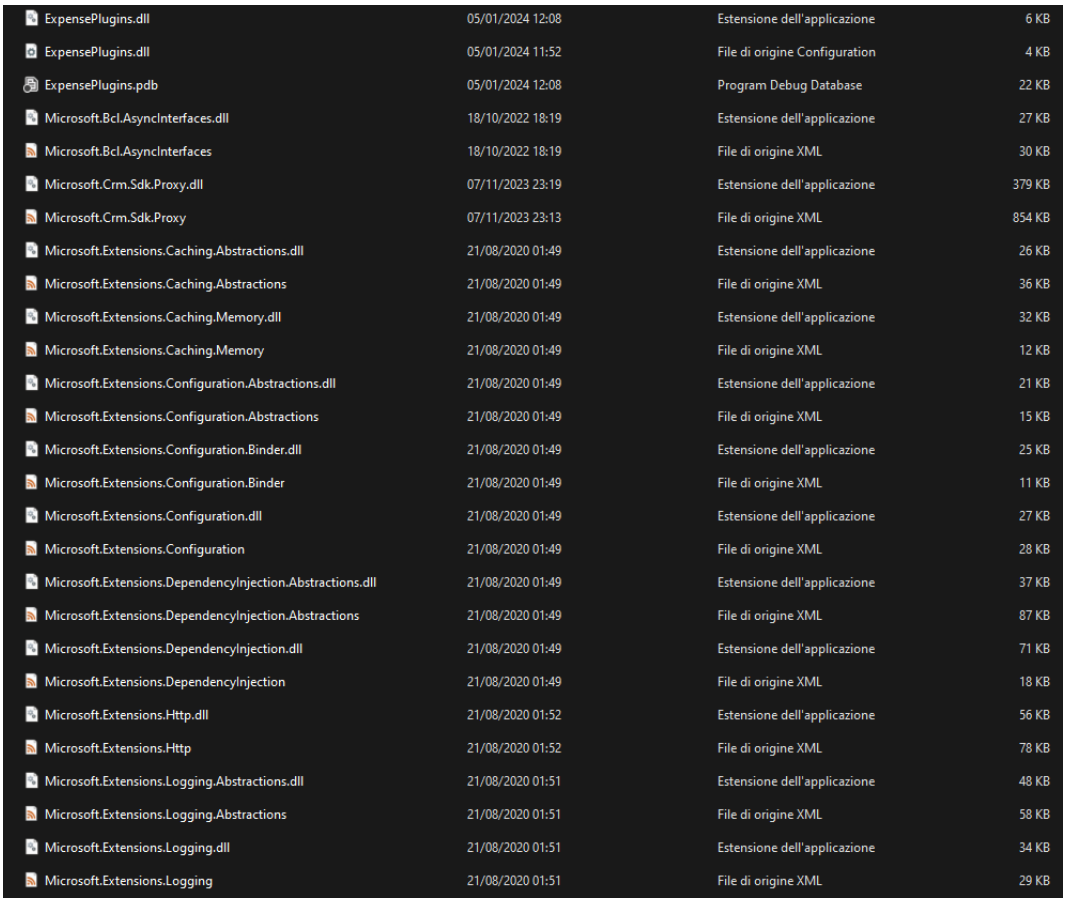
Finally, build the solution (CRTL+SHIFT+B). Remember that a rebuild of the solution is necessary each time a modification on the plugin is performed.

If we now open the ExpensePlugins folder



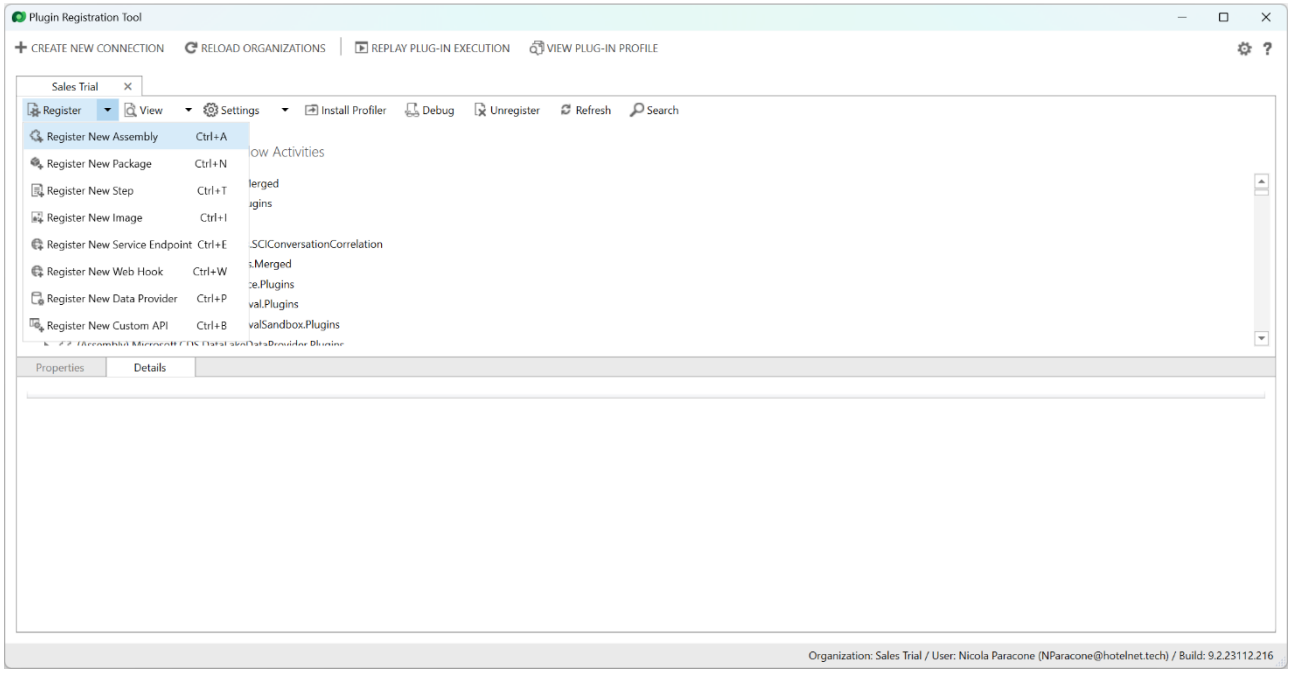
bin	05/01/2024 10:52	Cartella di file	
obj	05/01/2024 10:45	Cartella di file	
Properties	05/01/2024 10:45	Cartella di file	
app	05/01/2024 11:52	File di origine Configuration	4 KB
ExpensePlugins.csproj	05/01/2024 12:08	C# Project File	14 KB
key.snk	05/01/2024 12:07	Strong Name Key File	1 KB
packages	05/01/2024 11:52	File di origine Configuration	4 KB
PluginExample.cs	05/01/2024 11:52	C# Source File	3 KB

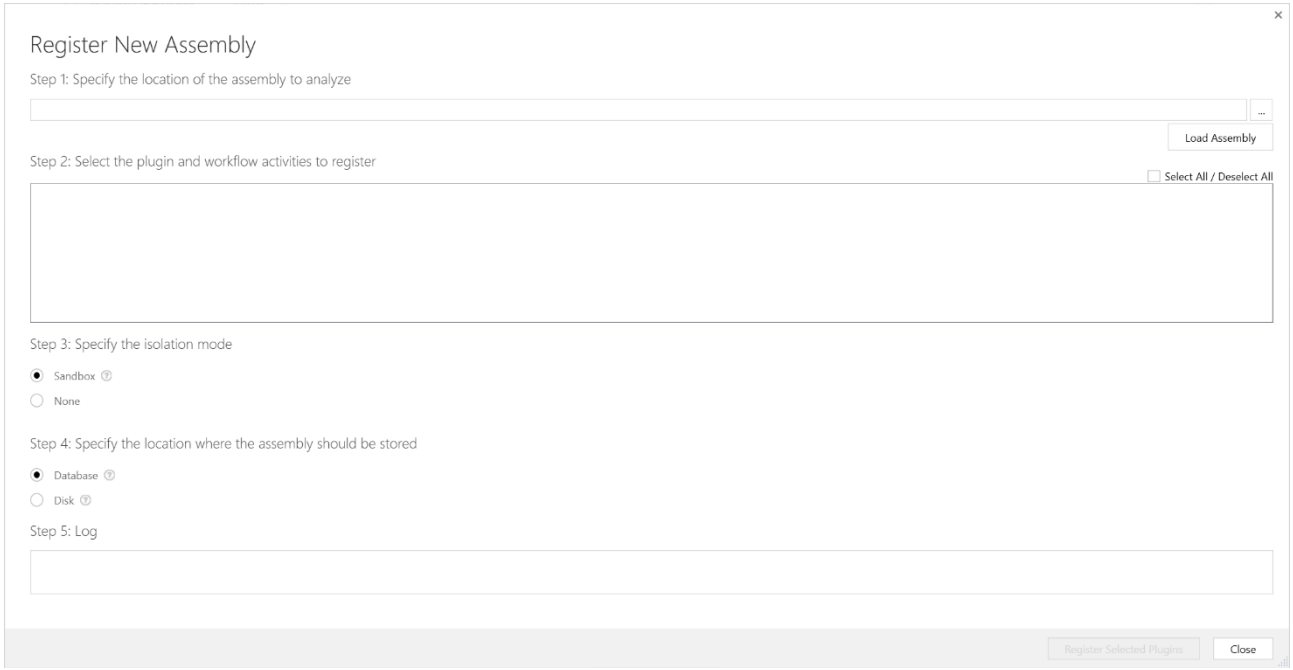
bin and then Debug, we will find a lot of dlls. ExpensePlugins.dll is the one we will use.



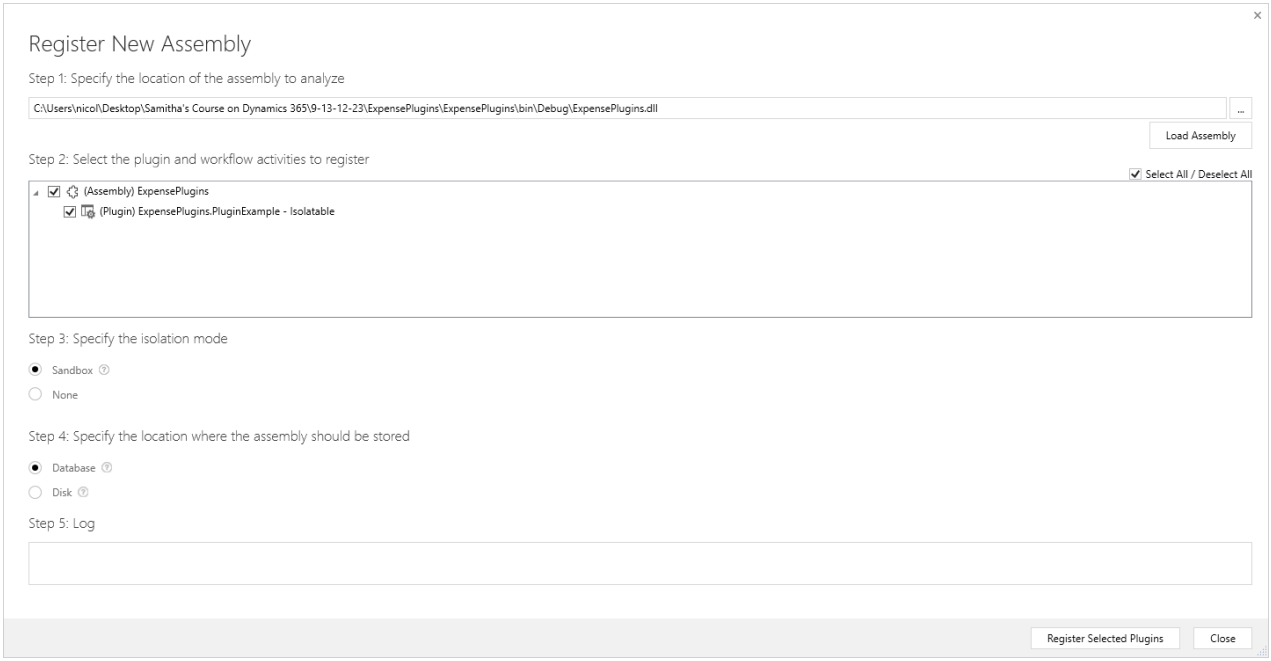
ExpensePlugins.dll	05/01/2024 12:08	Estensione dell'applicazione	6 KB
ExpensePlugins.dll	05/01/2024 11:52	File di origine Configuration	4 KB
ExpensePlugins.pdb	05/01/2024 12:08	Program Debug Database	22 KB
Microsoft.Bcl.AsyncInterfaces.dll	18/10/2022 18:19	Estensione dell'applicazione	27 KB
Microsoft.Bcl.AsyncInterfaces	18/10/2022 18:19	File di origine XML	30 KB
Microsoft.Crm.Sdk.Proxy.dll	07/11/2023 23:19	Estensione dell'applicazione	379 KB
Microsoft.Crm.Sdk.Proxy	07/11/2023 23:13	File di origine XML	854 KB
Microsoft.Extensions.Caching.Abstractions.dll	21/08/2020 01:49	Estensione dell'applicazione	26 KB
Microsoft.Extensions.Caching.Abstractions	21/08/2020 01:49	File di origine XML	36 KB
Microsoft.Extensions.Caching.Memory.dll	21/08/2020 01:49	Estensione dell'applicazione	32 KB
Microsoft.Extensions.Caching.Memory	21/08/2020 01:49	File di origine XML	12 KB
Microsoft.Extensions.Configuration.Abstractions.dll	21/08/2020 01:49	Estensione dell'applicazione	21 KB
Microsoft.Extensions.Configuration.Abstractions	21/08/2020 01:49	File di origine XML	15 KB
Microsoft.Extensions.Configuration.Binder.dll	21/08/2020 01:49	Estensione dell'applicazione	25 KB
Microsoft.Extensions.Configuration.Binder	21/08/2020 01:49	File di origine XML	11 KB
Microsoft.Extensions.Configuration.dll	21/08/2020 01:49	Estensione dell'applicazione	27 KB
Microsoft.Extensions.Configuration	21/08/2020 01:49	File di origine XML	28 KB
Microsoft.Extensions.DependencyInjection.Abstractions.dll	21/08/2020 01:49	Estensione dell'applicazione	37 KB
Microsoft.Extensions.DependencyInjection.Abstractions	21/08/2020 01:49	File di origine XML	87 KB
Microsoft.Extensions.DependencyInjection.dll	21/08/2020 01:49	Estensione dell'applicazione	71 KB
Microsoft.Extensions.DependencyInjection	21/08/2020 01:49	File di origine XML	18 KB
Microsoft.Extensions.Http.dll	21/08/2020 01:52	Estensione dell'applicazione	56 KB
Microsoft.Extensions.Http	21/08/2020 01:52	File di origine XML	78 KB
Microsoft.Extensions.Logging.Abstractions.dll	21/08/2020 01:51	Estensione dell'applicazione	48 KB
Microsoft.Extensions.Logging.Abstractions	21/08/2020 01:51	File di origine XML	58 KB
Microsoft.Extensions.Logging.dll	21/08/2020 01:51	Estensione dell'applicazione	34 KB
Microsoft.Extensions.Logging	21/08/2020 01:51	File di origine XML	29 KB

Back to the Plugin Registration Tool, we click on Register and then Register New Assembly.

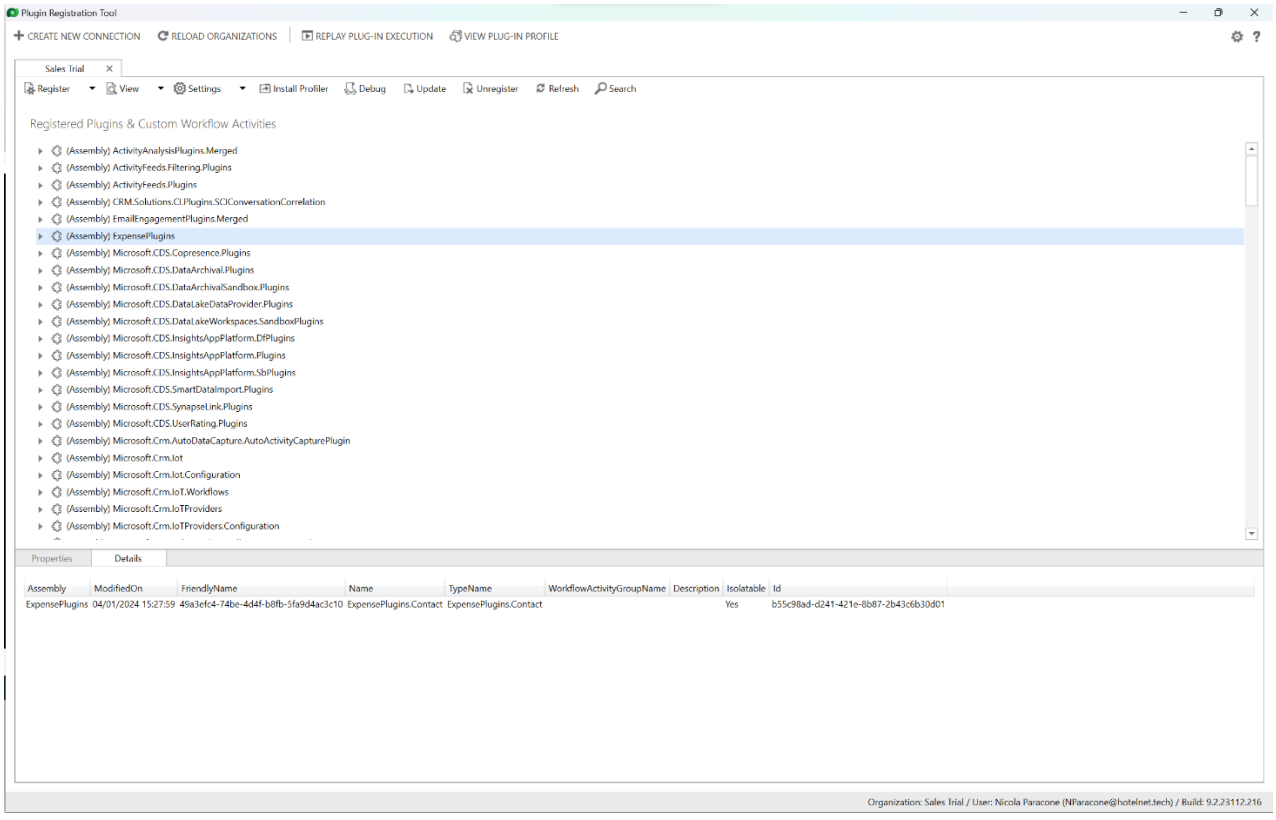




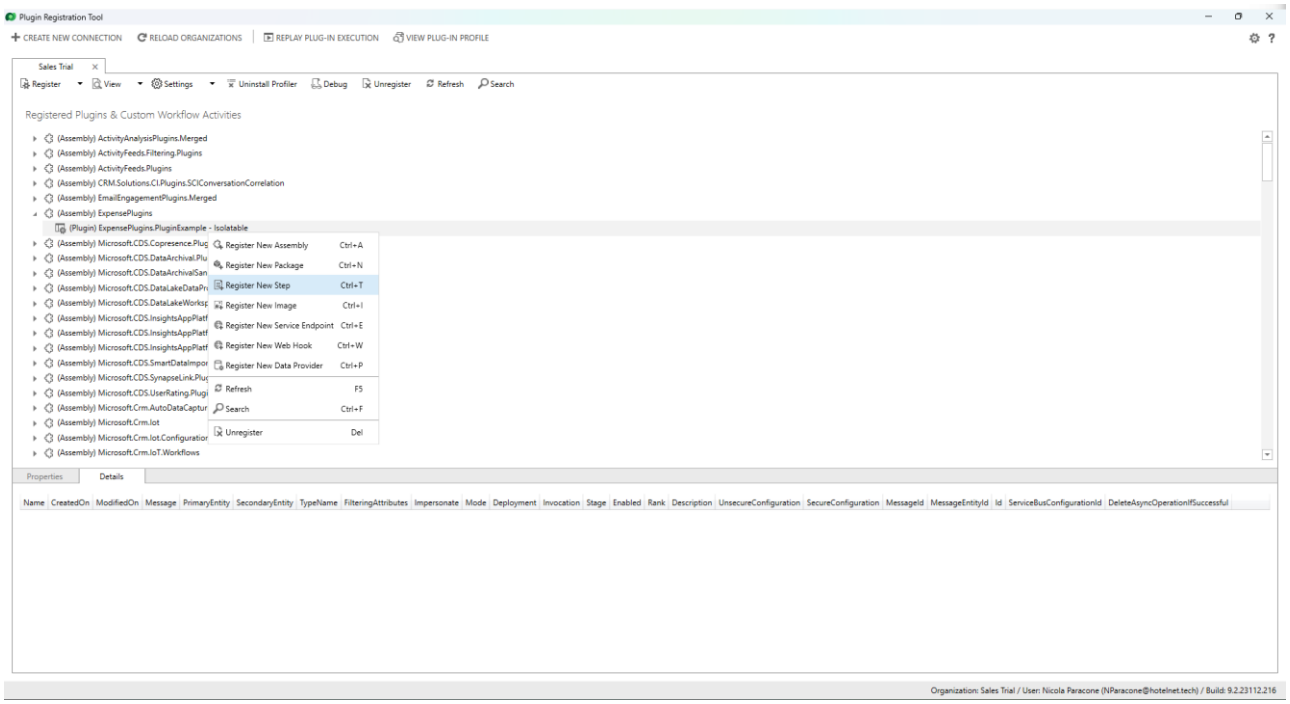
Click on the three dots, find the ExpensePlugins.dll we were talking about before and click on Load Assembly.



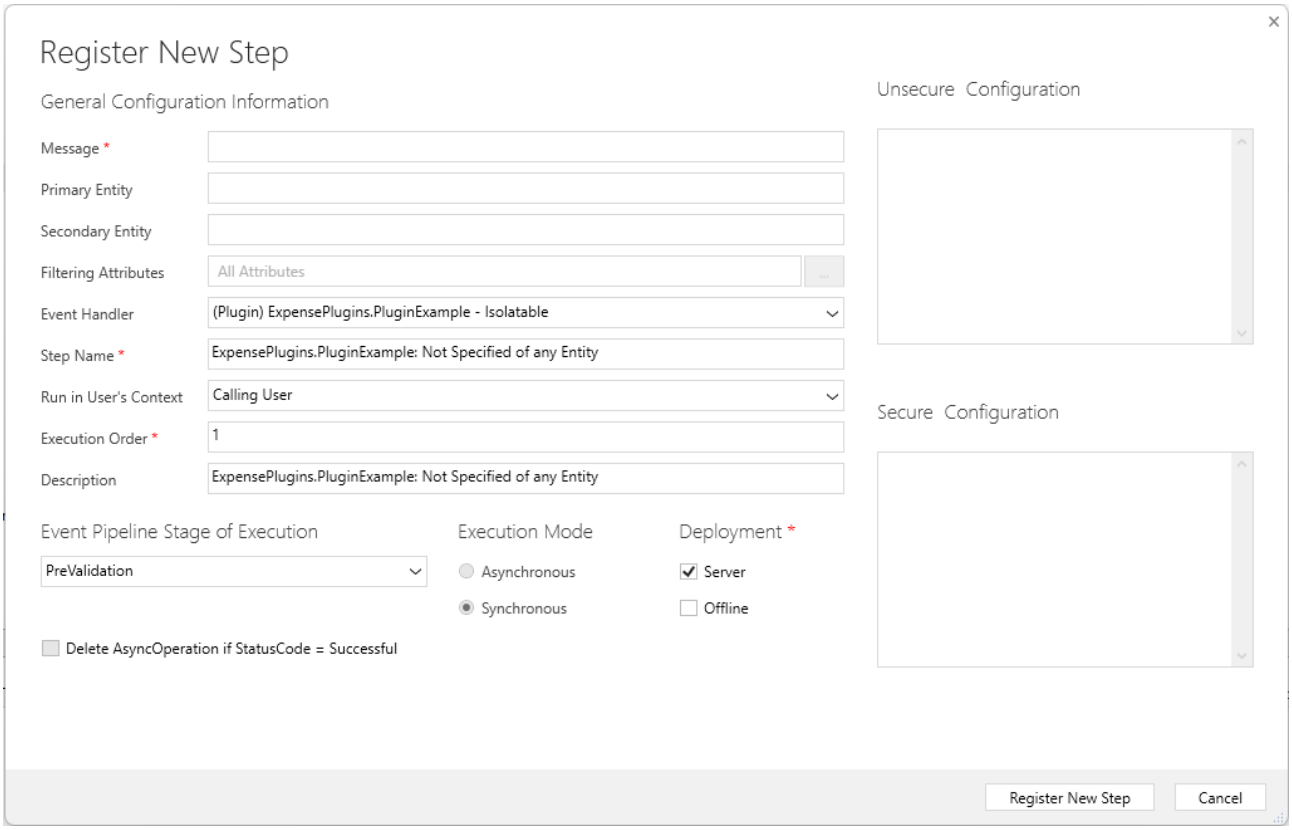
The plugin is now registered.



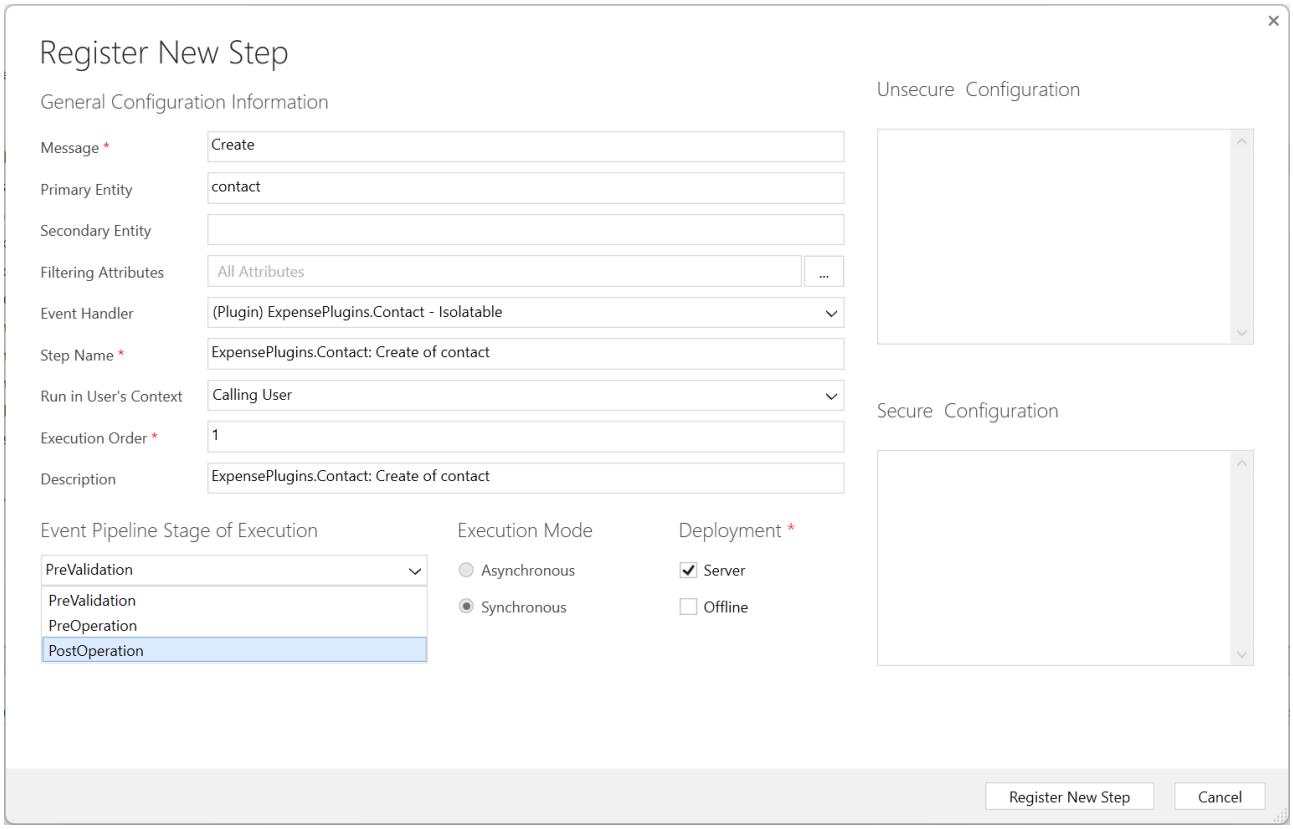
At this point we must specify when to trigger this plugin. For that we add steps.



Click on Register New Step.



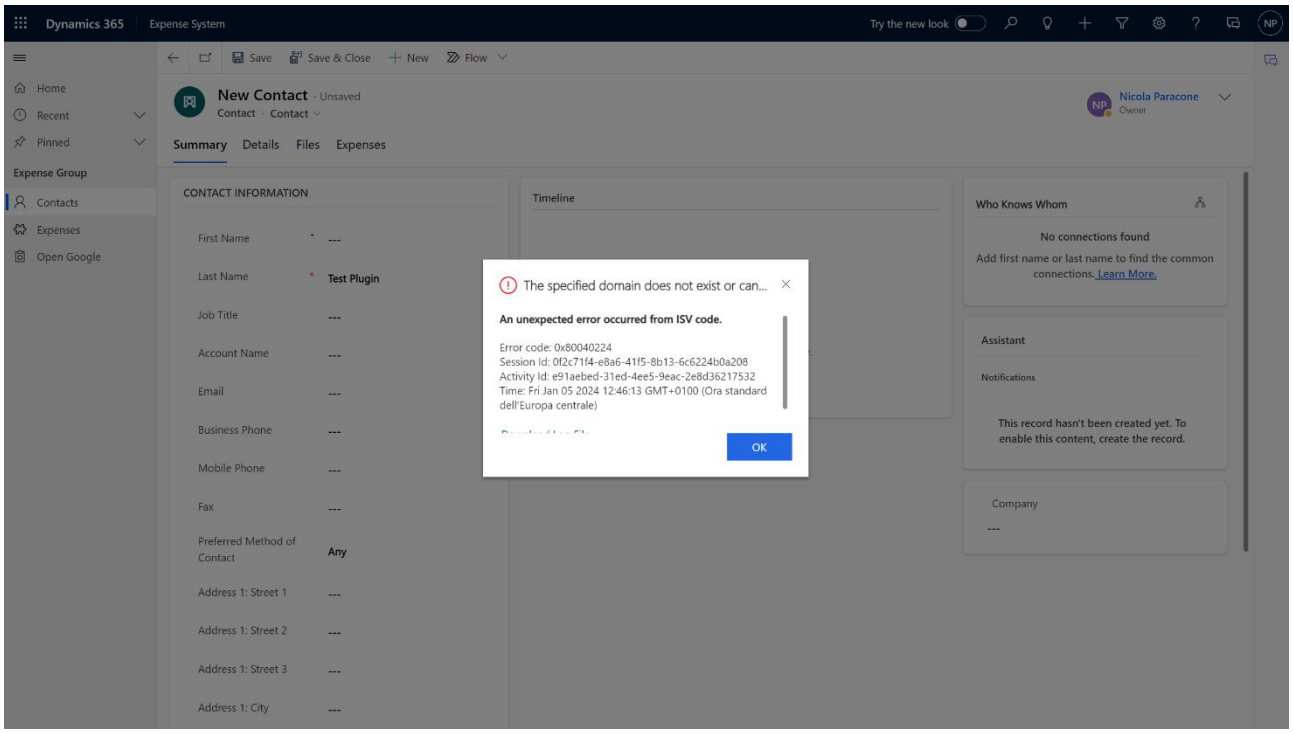
In the following we state that the plugin will run when a contact is created in PreValidation.



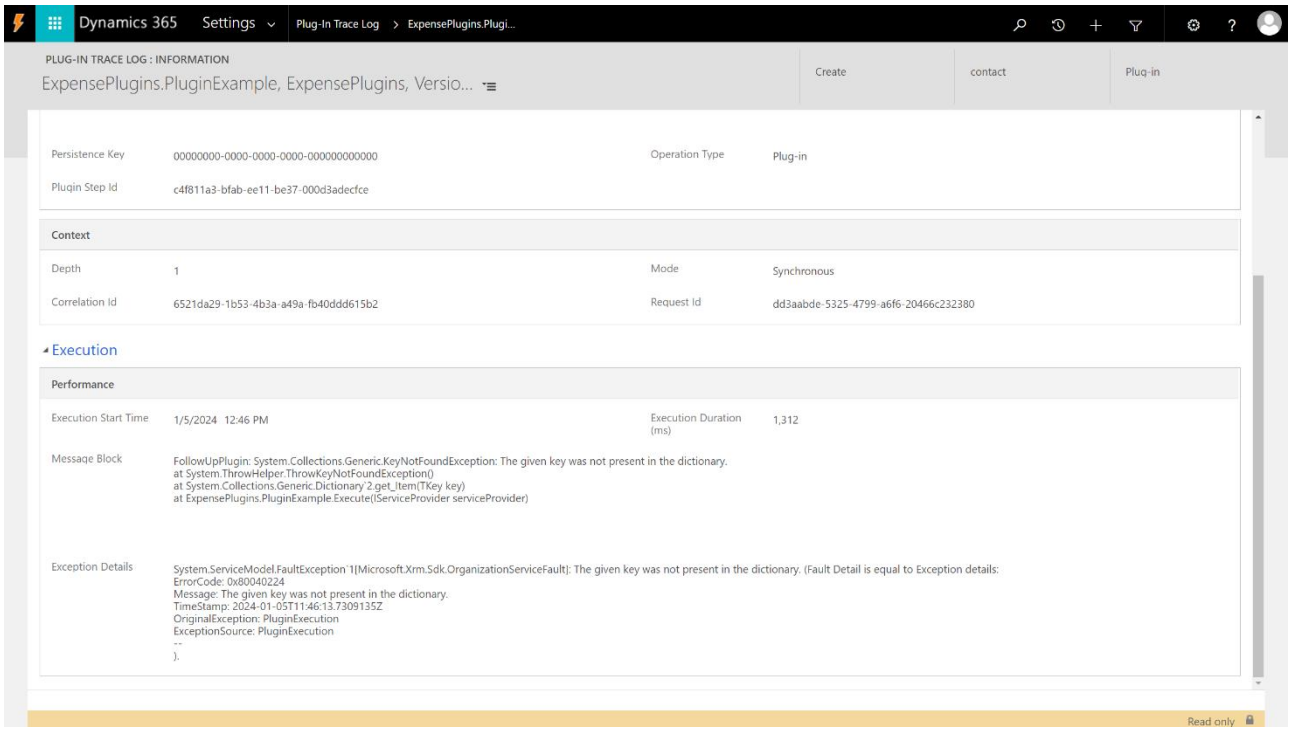
Click on Register New Step and let us test the functionality in Dynamics 365. Remember that we are only tracing the email.

2. Testing the plugin

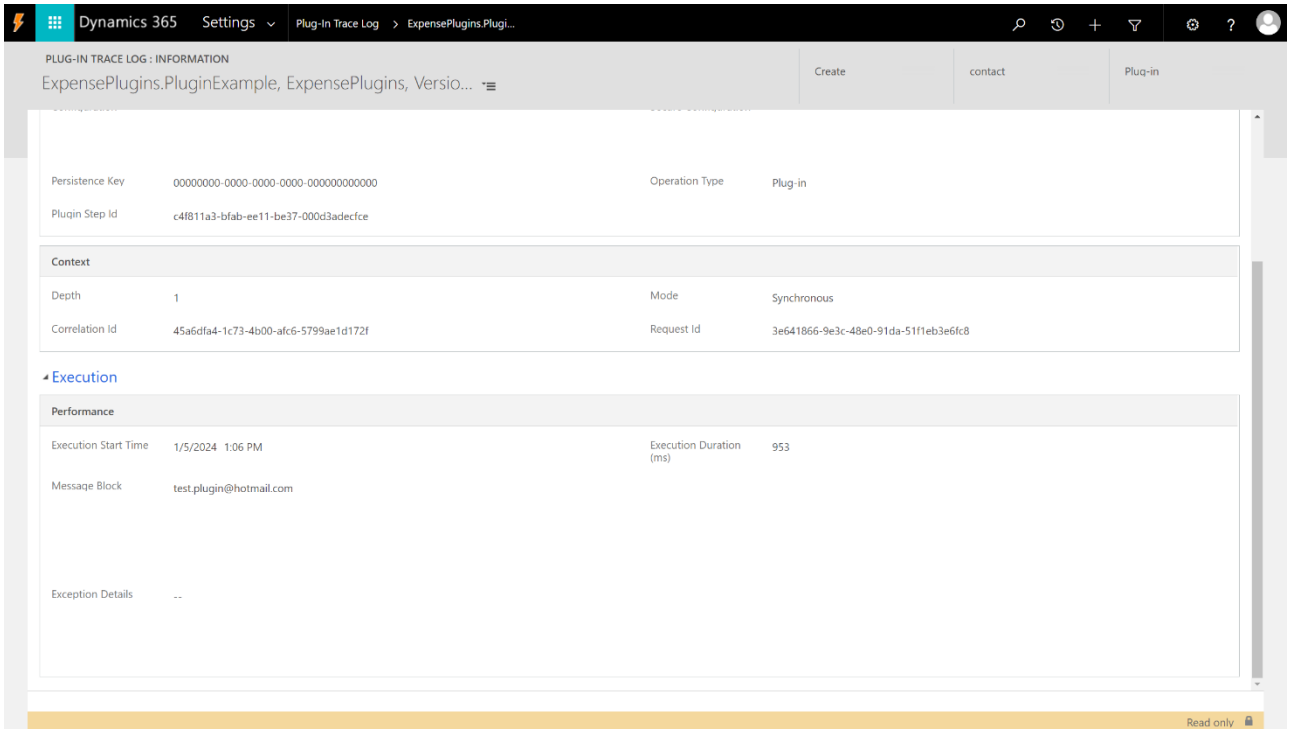
Let us create a new contact without providing an email.



Obviously we are thrown an error because we didn't tell to the plugin what to do when the email field is empty.



In creating a contact with an email associated, instead, everything is fine and the email is correctly traced.



To avoid this error, we simply modify the plugin code as follows.

```
try
{
    if (entity.LogicalName == "contact")
    {
        if (entity.Attributes.Contains("emailaddress1"))
        {

            string email = entity.Attributes["emailaddress1"].ToString();
            tracingService.Trace(email);
        }

    }
}
```

Now we will trigger the plugin on an expense creation. When a new expense is created, we want to get the contact and check if the email is null or contains data. If the email is null, we want an error to be thrown to the user.

Contact is a lookup field inside an expense. Therefore, it must be referenced using EntityReference:

```
EntityReference contactId = (EntityReference)entity.Attributes["demo_contact"];
Guid contactGuid = contactId.Id;
```

If we were working with option set values, we would type something like:

```
int option = Convert.ToInt32((OptionSetValue)entity.Attributes["demo_expensetype"]);
```

Working with currency values, we would have:

```
Money amount = (Money)entity.Attributes["demo_expenseamount"];
```

For a yes or no field:

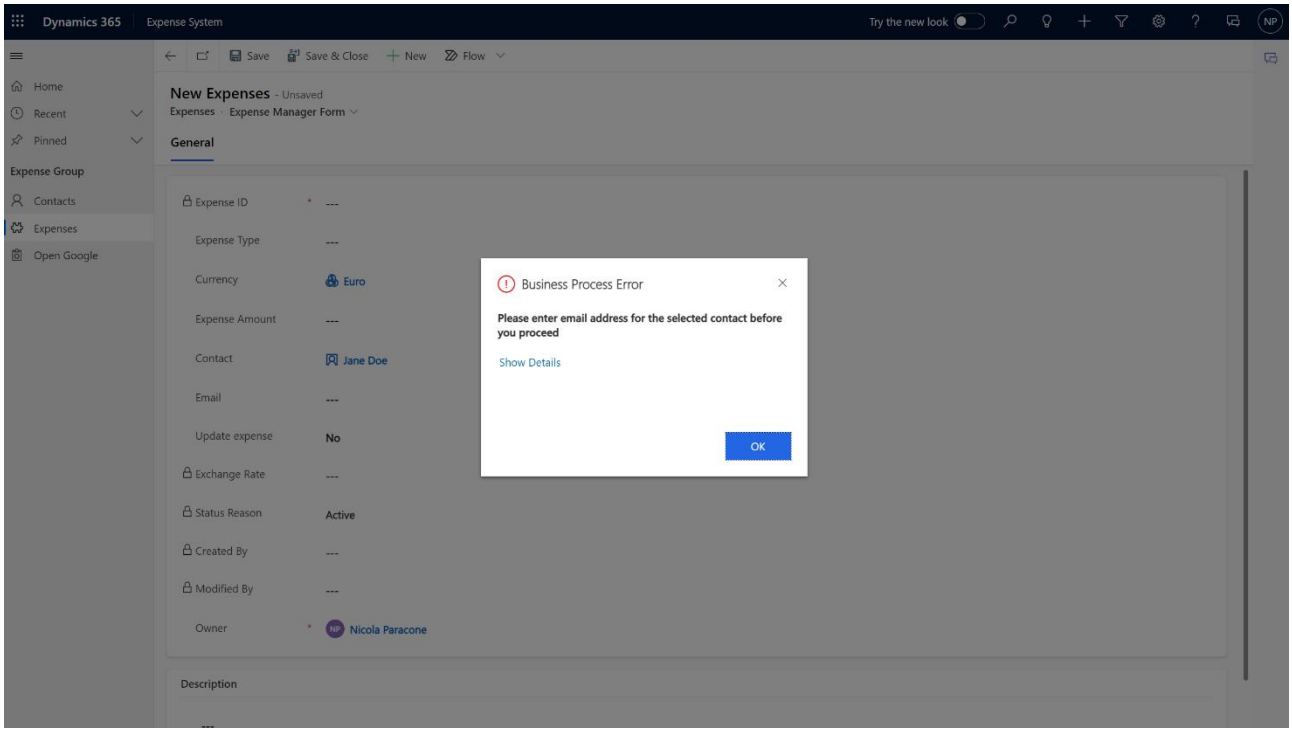
```
bool yesorno = (bool)entity.Attributes["demo_yesorno"];
```

Let us modify the try block as follows.

```
try
{
    if (entity.LogicalName == "demo_expenses")
    {
        if (entity.Attributes.Contains("demo_contact"))
        {
            tracingService.Trace("Contact has value");

            EntityReference contactId = (EntityReference)entity.Attributes["demo_contact"];
            tracingService.Trace(contactId.Id.ToString());

            Entity contactdata = service.Retrieve("contact", contactId.Id, new ColumnSet("emailaddress1"));
            tracingService.Trace("contactdata retrieved");

            if (contactdata.Attributes.Contains("emailaddress1") && contactdata.Attributes["emailaddress1"] != null)
            {
                tracingService.Trace("email has value");
            }
            else
            {
                tracingService.Trace("email is null");
                throw new InvalidPluginExecutionException("Please enter email address for the selected contact before you proceed");
            }
        }

    }
}
```

Register the new plugin on creation of a new expense.

To test it, we create a new expense and set the value of Contact to Jane Doe, which has no email address. This is what we get when we try to save the record, namely the error message we have inserted in the plugin code.

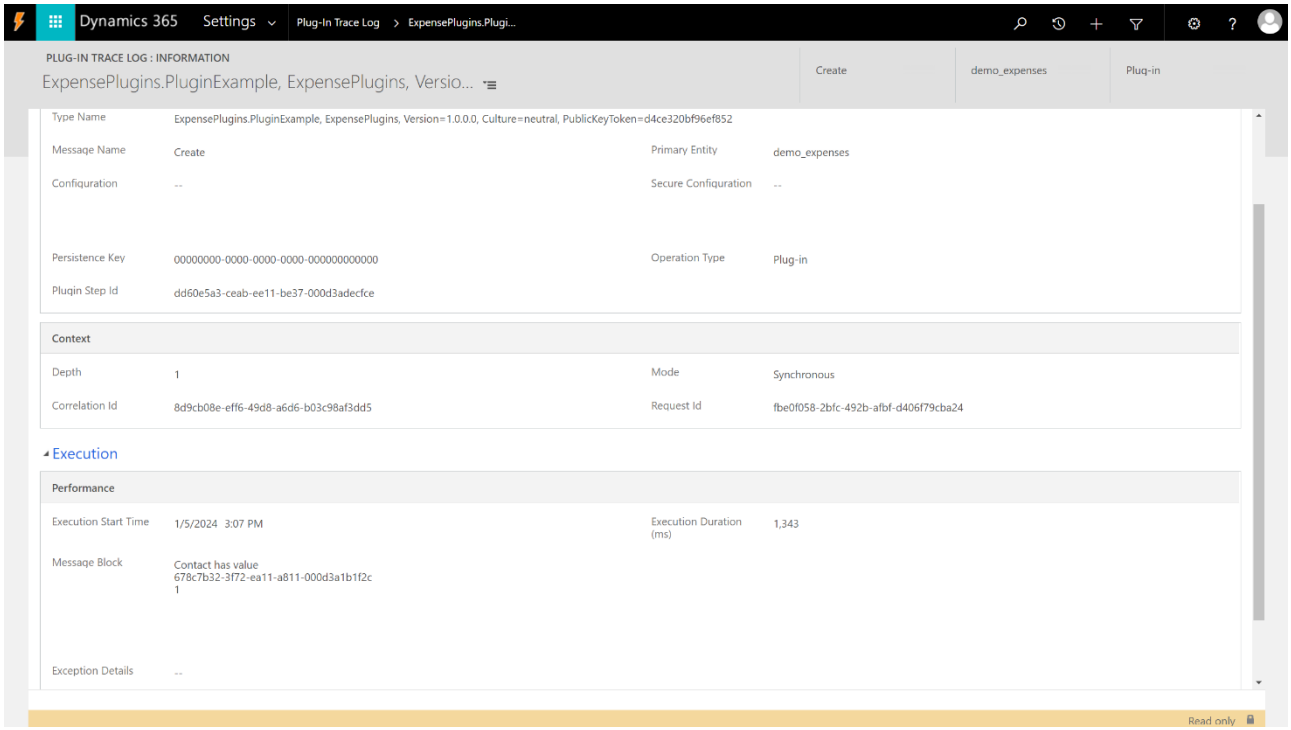


Let us explore a further example: whenever an expense is created we want to retrieve the contact Id and then retrieve the count of all the expenses associated with that contact.

This is obtained by means of the following code, which uses a FetchXML string.

```
            try
            {
                if (entity.LogicalName == "demo_expenses")
                {
                    if (entity.Attributes.Contains("demo_contact"))
                    {
                        tracingService.Trace("Contact has value");

                        EntityReference contactId = (EntityReference)entity.Attributes["demo_contact"];
                        tracingService.Trace(contactId.Id.ToString());

                        string fetchxml = "<fetch version='1.0' output-format='xml-platform' mapping='logical' distinct='false'>" +
"<entity name='demo_expenses'>" +
"<attribute name='demo_expensesid' />" +
"<attribute name='demo_name' />" +
"<attribute name='createdon' />" +
"<filter type='and'>" +
"<condition attribute='demo_contact' operator='eq' value='" + contactId.Id+"' uitype='contact' />" +
"</filter>" +
"</entity>" +
"</fetch>";



                        EntityCollection expenses = service.RetrieveMultiple(new FetchExpression(fetchxml));
                        tracingService.Trace(expenses.Entities.Count.ToString());
                    }

                }
            }
```

Kevin Martin, for example, owns zero expenses. So, when we create a new expense associated with him, the log file produced will be the following.



Keep in mind that the plugin step has been registered on PostOperation.

Since we retrieve a collection of expenses, we can also add a foreach block inside our code to display the expense amount.

```
EntityCollection expenses = service.RetrieveMultiple(new FetchExpression(fetchxml));
tracingService.Trace(expenses.Entities.Count.ToString());
foreach(Entity item in expenses.Entities)
{
    tracingService.Trace(item.Attributes["demo_expenseamount"].ToString());
}
```

3. Query expressions to retrieve data

We already learnt how to create, update, retrieve and delete records using console apps. We can do the same operations using plugins by recycling our existing code. Usually, a common strategy consists first in writing the code inside a console application and then, once we are sure that it is properly working, put it into a plugin.

An alternative method to using a FetchXML string, when we want to retrieve data, is by means of a **Query expression**.

When it comes to Query expressions, however, we cannot directly test the correctness of the data we are querying. This operation was simple in the case of using the FetchXML string since, before downloading the FetchXML file, we have a preview of the records directly in Dynamics 365.

Save the following link https://learn.microsoft.com/en-us/power-apps/developer/data-platform/org-service/entity-operations for an introduction to the Entity class.

For an introduction to the use of Query expressions, instead, navigate to https://learn.microsoft.com/en-us/power-apps/developer/data-platform/org-service/entity-operations-query-data.

The following example shows a simple query to return up to 50 matching account rows where the address1_city value equals Redmond, ordered by name.

```
var query = new QueryExpression("account")
{
  ColumnSet = new ColumnSet("name"),
  Criteria = new FilterExpression(LogicalOperator.And),
  TopCount = 50
};
query.Criteria.AddCondition("address1_city", ConditionOperator.Equal, "Redmond");
query.AddOrder("name", OrderType.Ascending);

EntityCollection results = svc.RetrieveMultiple(query);

results.Entities.ToList().ForEach(x =>
{
  Console.WriteLine(x.Attributes["name"]);
});
```

Unlike FetchXML and QueryExpression, QueryByAttribute can only return data from a single table. It doesn't enable retrieving data from related table rows or complex query criteria.

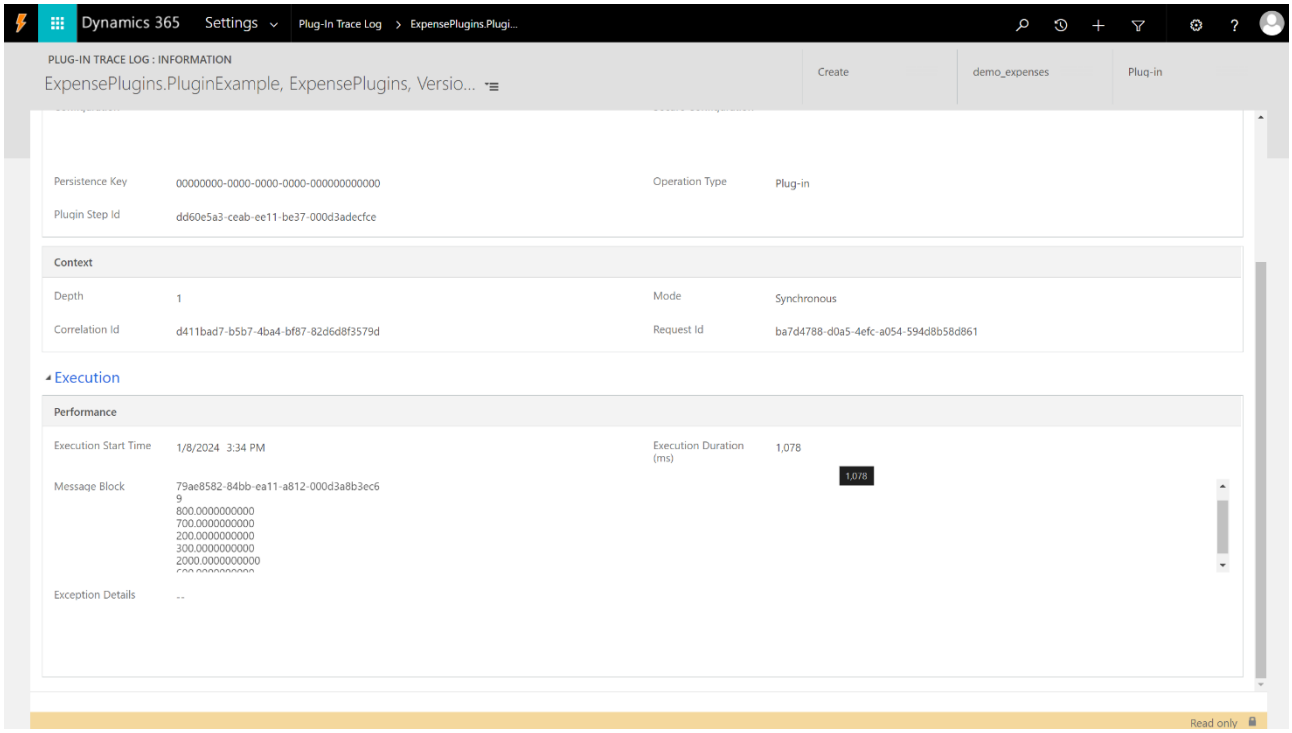
```
var query = new QueryByAttribute("account")
{
  TopCount = 50,
  ColumnSet = new ColumnSet("name")
};
query.AddAttributeValue("address1_city", "Redmond");
query.AddOrder("name", OrderType.Ascending);

EntityCollection results = svc.RetrieveMultiple(query);

results.Entities.ToList().ForEach(x =>
{
  Console.WriteLine(x.Attributes["name"]);
});
```

Consider the following try block implemented in our previous plugin.

```
try
{
    if (entity.LogicalName == "demo_expenses")
    {
        if (entity.Attributes.Contains("demo_contact"))
        {
            tracingService.Trace("Contact has value");

            EntityReference contactId = (EntityReference)entity.Attributes["demo_contact"];
            tracingService.Trace(contactId.Id.ToString());

            QueryExpression query1 = new QueryExpression("demo_expenses");
            query1.ColumnSet = new ColumnSet("demo_expenseamount");
            query1.Criteria.AddCondition("demo_contact", ConditionOperator.Equal,contactId.Id);

            EntityCollection expenses = service.RetrieveMultiple(query1);
            tracingService.Trace(expenses.Entities.Count.ToString());
            foreach (Entity item in expenses.Entities)
            {
                if (item.Attributes.Contains("demo_expenseamount") && item.Attributes["demo_expenseamount"] != null)
                {
                    Money expensevalue = (Money)item.Attributes["demo_expenseamount"];
                    tracingService.Trace(expensevalue.Value.ToString());

                }
            }
        }

    }
}
```

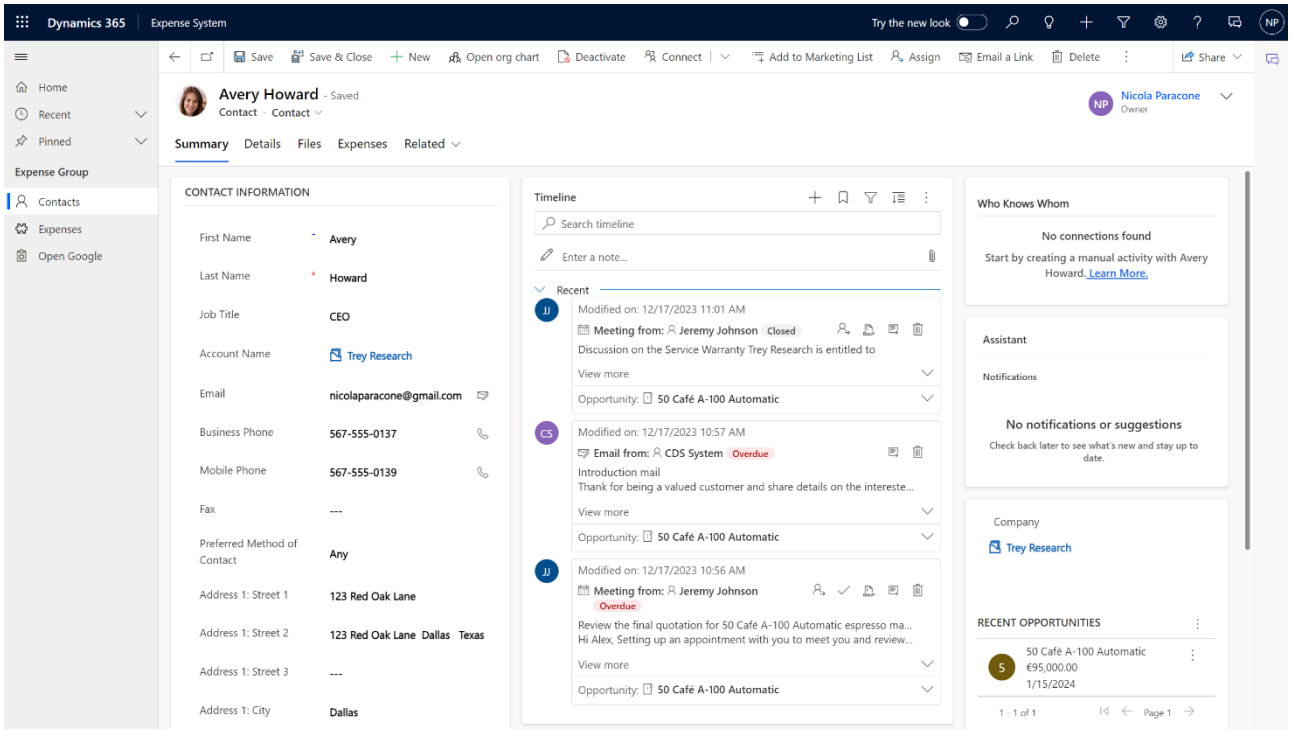
The action of this plugin is the following: when an expense is created, the Guid of the associated contact is retrieved; then, by means of a Query expression the plugin retrieves the values of Expense Amount in each of the expenses associated with that contact and traces them into a log.

So, if we create a new expense for Avery Howard, we will get the following trace log.



Of nine expenses associated with Avery Howard, six contain a non-null expense amount.

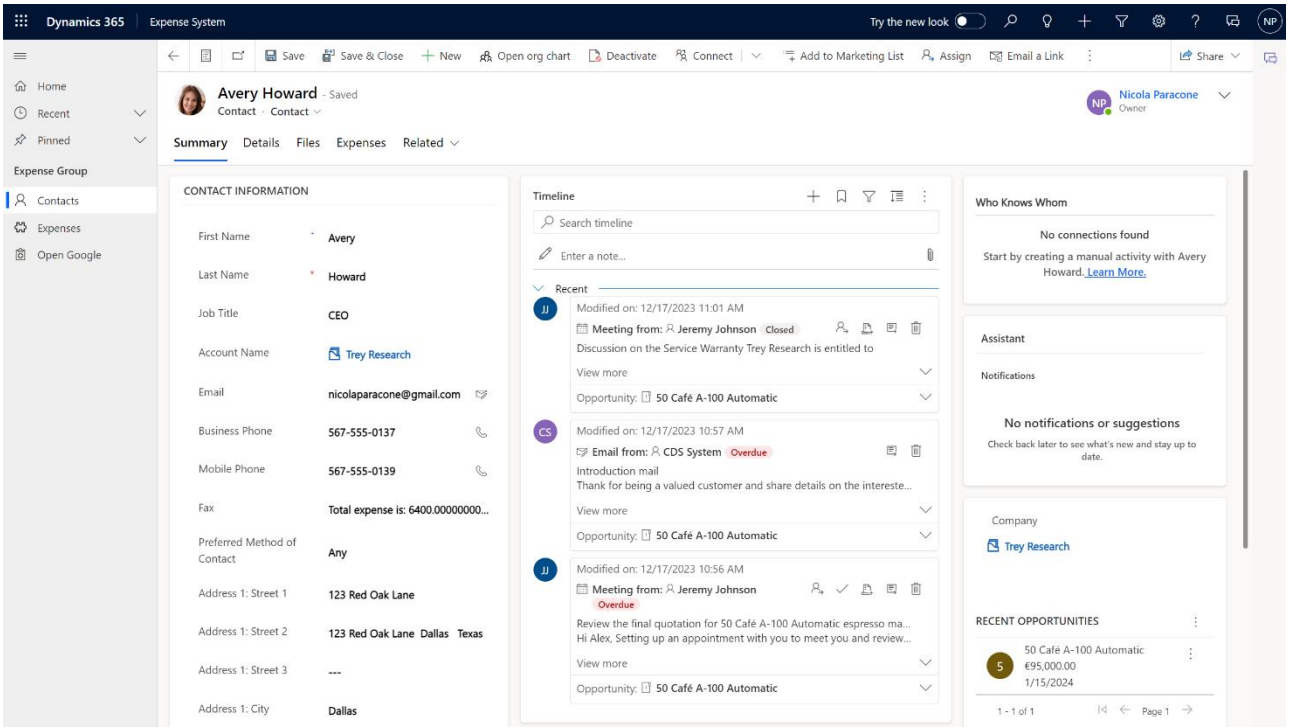
Suppose now that, for some reason, we want to update the content of Fax for an account with the total expense amount.



A code which we can use is the following.

```
try
{
    if (entity.LogicalName == "demo_expenses")
    {
        if (entity.Attributes.Contains("demo_contact"))
        {
            tracingService.Trace("Contact has value");

            EntityReference contactId = (EntityReference)entity.Attributes["demo_contact"];
            tracingService.Trace(contactId.Id.ToString());

            QueryExpression query1 = new QueryExpression("demo_expenses");
            query1.ColumnSet = new ColumnSet("demo_expenseamount");
            query1.Criteria.AddCondition("demo_contact", ConditionOperator.Equal,contactId.Id);

            EntityCollection expenses = service.RetrieveMultiple(query1);
            tracingService.Trace(expenses.Entities.Count.ToString());
            decimal amount = 0;
            foreach (Entity item in expenses.Entities)
            {
                if (item.Attributes.Contains("demo_expenseamount") && item.Attributes["demo_expenseamount"] != null)
                {
                    Money expensevalue = (Money)item.Attributes["demo_expenseamount"];
                    amount = amount + expensevalue.Value;
                    tracingService.Trace(expensevalue.Value.ToString());
                }
            }

            Entity contactUpdate = new Entity("contact");
            contactUpdate.Id = contactId.Id;
            contactUpdate["fax"] = "Total expense is: " + amount.ToString();
            service.Update(contactUpdate);
        }

    }
}
```

This will have the effect displayed in the figure below.



A simple way of debugging a plugin is by tracing the variables it uses. Alternatively, the Plugin Registration Tool provides a profiler and a debugging tool.

When we are using FetchXML or Query expressions, only the top 5000 records are retrieved. If we have to deal with larger amounts of data, we need to use another piece of code.

This is extensively described at:

https://learn.microsoft.com/en-us/power-apps/developer/data-platform/org-service/page-large-result-sets-with-fetchxml

for FetchXML, and at:

https://learn.microsoft.com/en-us/power-apps/developer/data-platform/org-service/page-large-result-sets-with-queryexpression

for QueryExpression.

Check out the following code, which performs exactly the same operations as the previous one but allowing to retrieve more than the default number of records.

```
try
{
    if (entity.LogicalName == "demo_expenses")
    {
        if (entity.Attributes.Contains("demo_contact"))
        {
            tracingService.Trace("Contact has value");

            EntityReference contactId = (EntityReference)entity.Attributes["demo_contact"];
            tracingService.Trace(contactId.Id.ToString());

            int pageCount = 5000;

            // Initialize the page number.
            int pageNumber = 1;

            // Initialize the number of records.
            int recordCount = 0;

            QueryExpression query1 = new QueryExpression("demo_expenses");
            query1.ColumnSet = new ColumnSet("demo_expenseamount");
            query1.Criteria.AddCondition("demo_contact", ConditionOperator.Equal,contactId.Id);

            query1.PageInfo = new PagingInfo();
            query1.PageInfo.Count = pageCount;
            query1.PageInfo.PageNumber = pageNumber;
            query1.PageInfo.PagingCookie = null;

            EntityCollection results1 = new EntityCollection();

            while (true)
            {
                // Retrieve the page of 5000
                EntityCollection results = service.RetrieveMultiple(query1);

                // Adding the retrieved records to the entity collection
                results1.Entities.AddRange(results.Entities);
                // Check for more records, if it returns true.
                if (results.MoreRecords)
                {
                    // Increment the page number to retrieve the next page.
                    query1.PageInfo.PageNumber++;

                    // Set the paging cookie to the paging cookie returned from current results.
                    query1.PageInfo.PagingCookie = results.PagingCookie;
                }
                else
                {
                    // If no more records are in the result nodes, exit the loop.
                    break;
                }
            }


            tracingService.Trace(results1.Entities.Count.ToString());
            decimal amount = 0;
            foreach (Entity item in results1.Entities)
            {
                if (item.Attributes.Contains("demo_expenseamount") && item.Attributes["demo_expenseamount"] != null)
                {
                    Money expensevalue = (Money)item.Attributes["demo_expenseamount"];
                    amount = amount + expensevalue.Value;
                    tracingService.Trace(expensevalue.Value.ToString());

                }
            }

            Entity contactUpdate = new Entity("contact");
            contactUpdate.Id = contactId.Id;
            contactUpdate["fax"] = "Total expense is: " + amount.ToString();
            service.Update(contactUpdate);
        }

    }
}
```

4. Recovering source code from dll

XRM Toolbox is a set of tools designed to assist in customizing, configuring, and administering Microsoft Dynamics 365 and the Common Data Service (CDS). It is a popular tool among Dynamics 365 developers, administrators, and consultants. Some of the features and capabilities of XRM Toolbox include:

1. **Plugin and workflow management**: allows developers to create, import, export, and manage plugins and workflows within Dynamics 365.

2. **Metadata editing**: provides tools for modifying metadata within Dynamics 365, such as entities, attributes, and relationships.

3. **Data manipulation**: enables users to perform various data-related tasks, such as importing/exporting data, bulk updates, and data cleansing.

4. **Security configuration**: helps administrators manage security roles, permissions, and access levels within Dynamics 365.

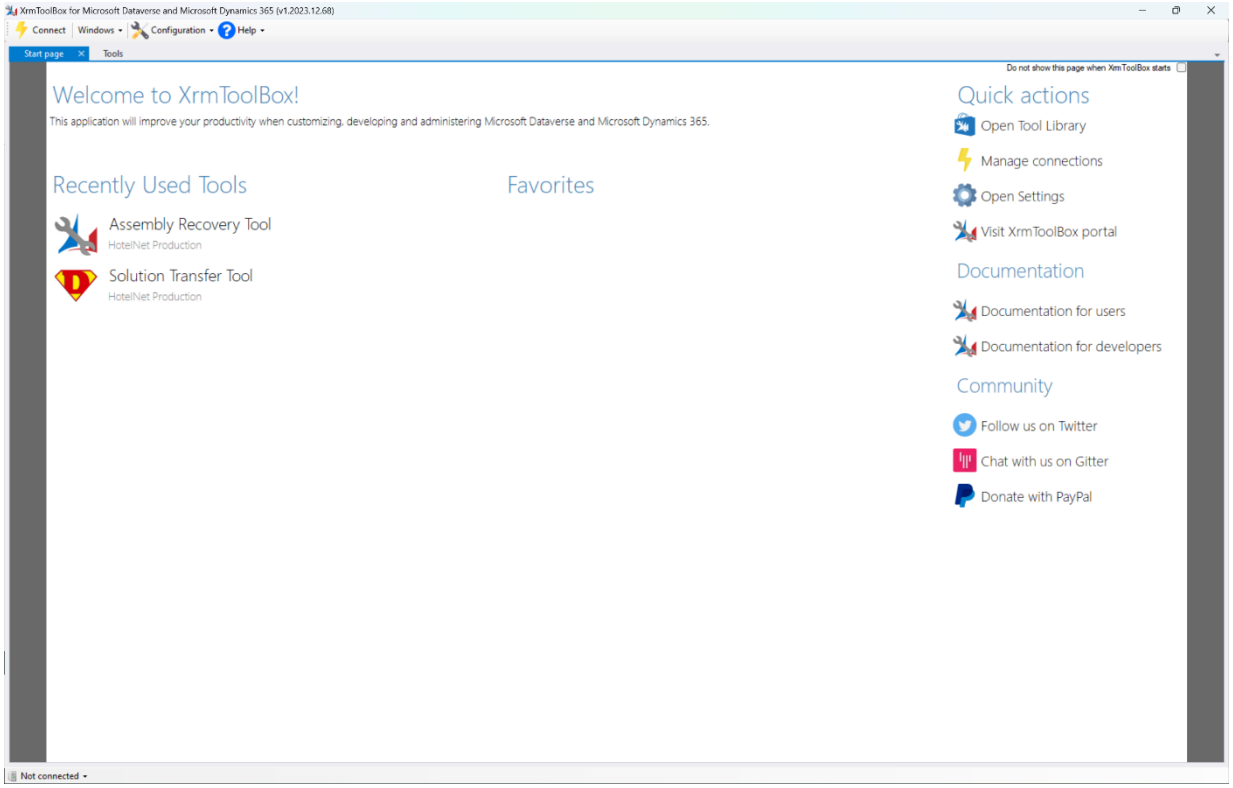
5. **Solution management**: facilitates the creation, deployment, and management of solutions, which are bundles of customizations in Dynamics 365.

6. **Web resources management**: allows users to manage web resources, such as HTML, JavaScript, and CSS files, which are used in customizations and extensions.

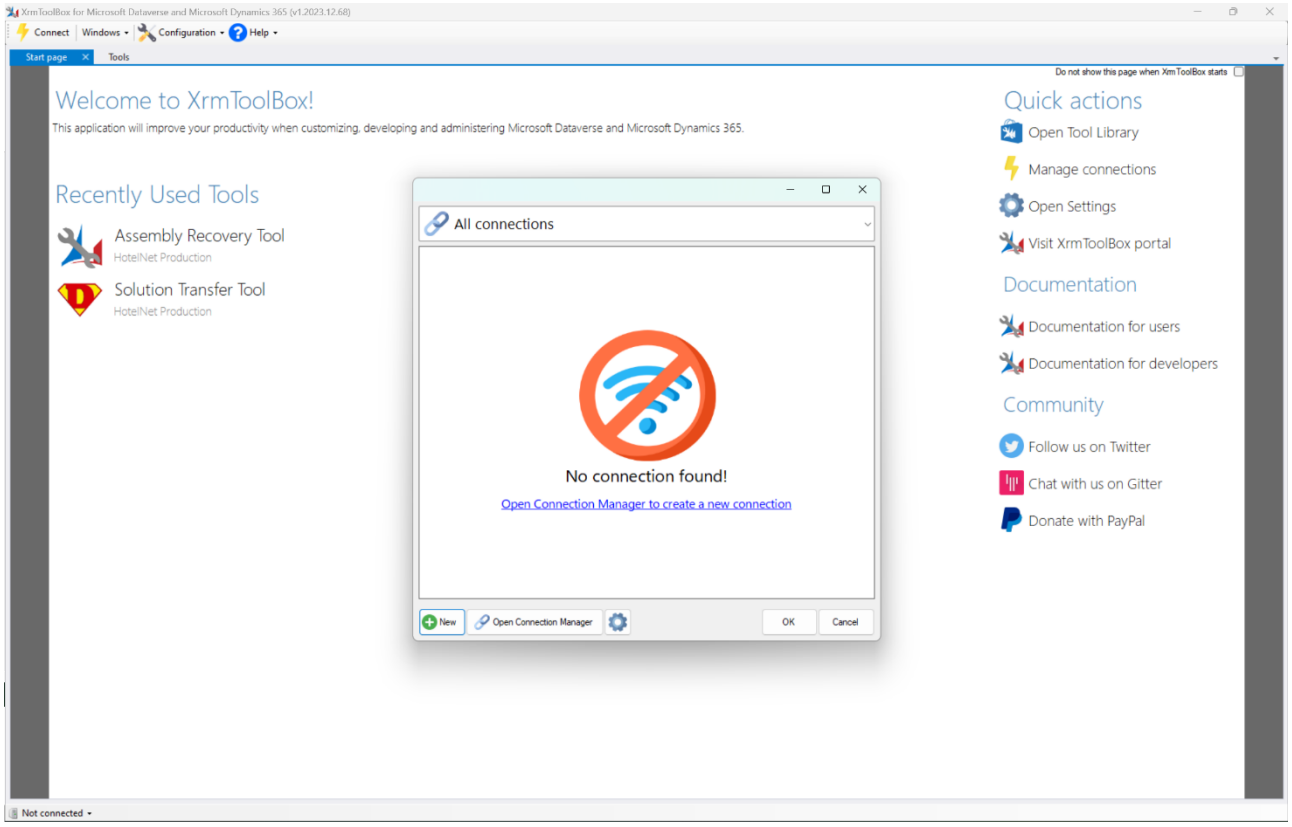
7. **Trace log analysis**: provides tools for analyzing trace logs generated by Dynamics 365 to troubleshoot issues and performance problems.

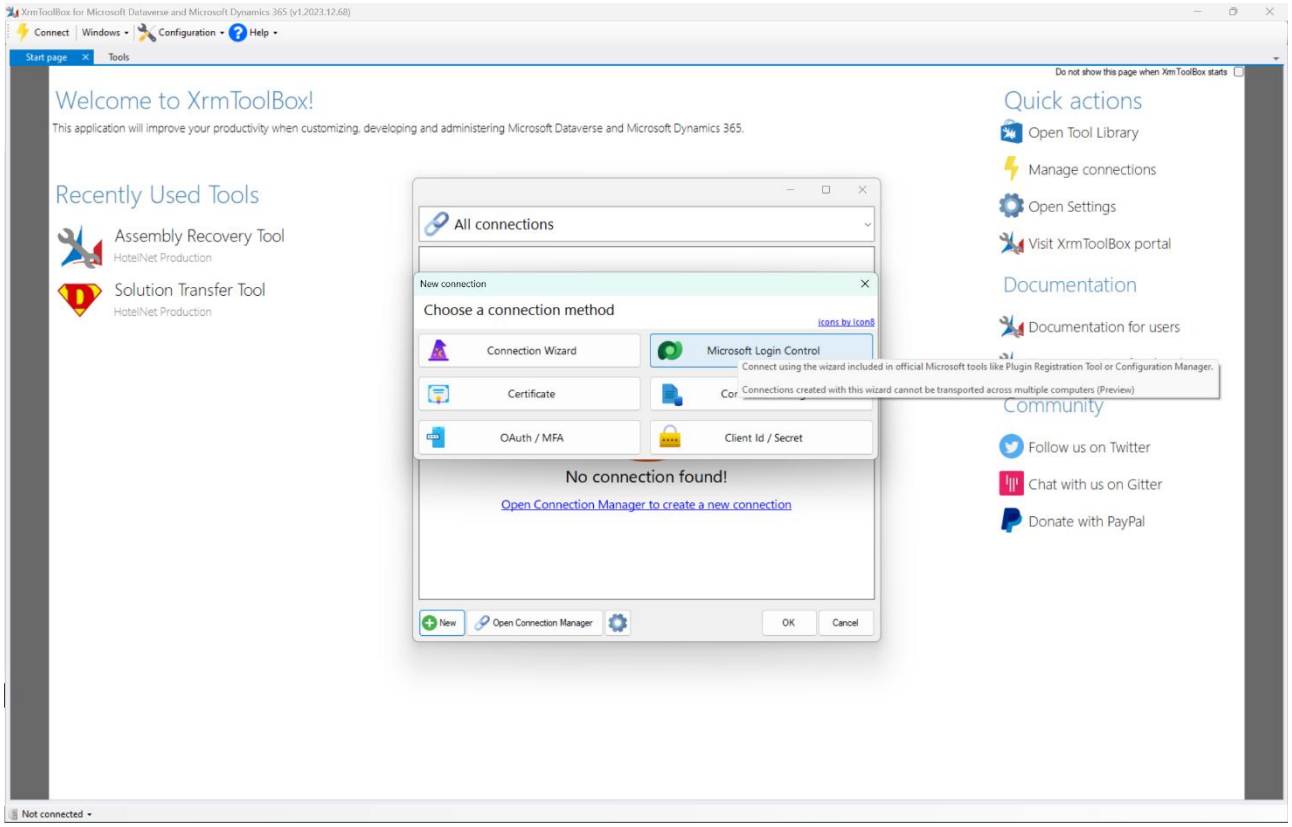
Overall, XRM Toolbox is a versatile and powerful tool that enhances the capabilities of Dynamics 365 and makes it easier for developers and administrators to customize and manage the platform.

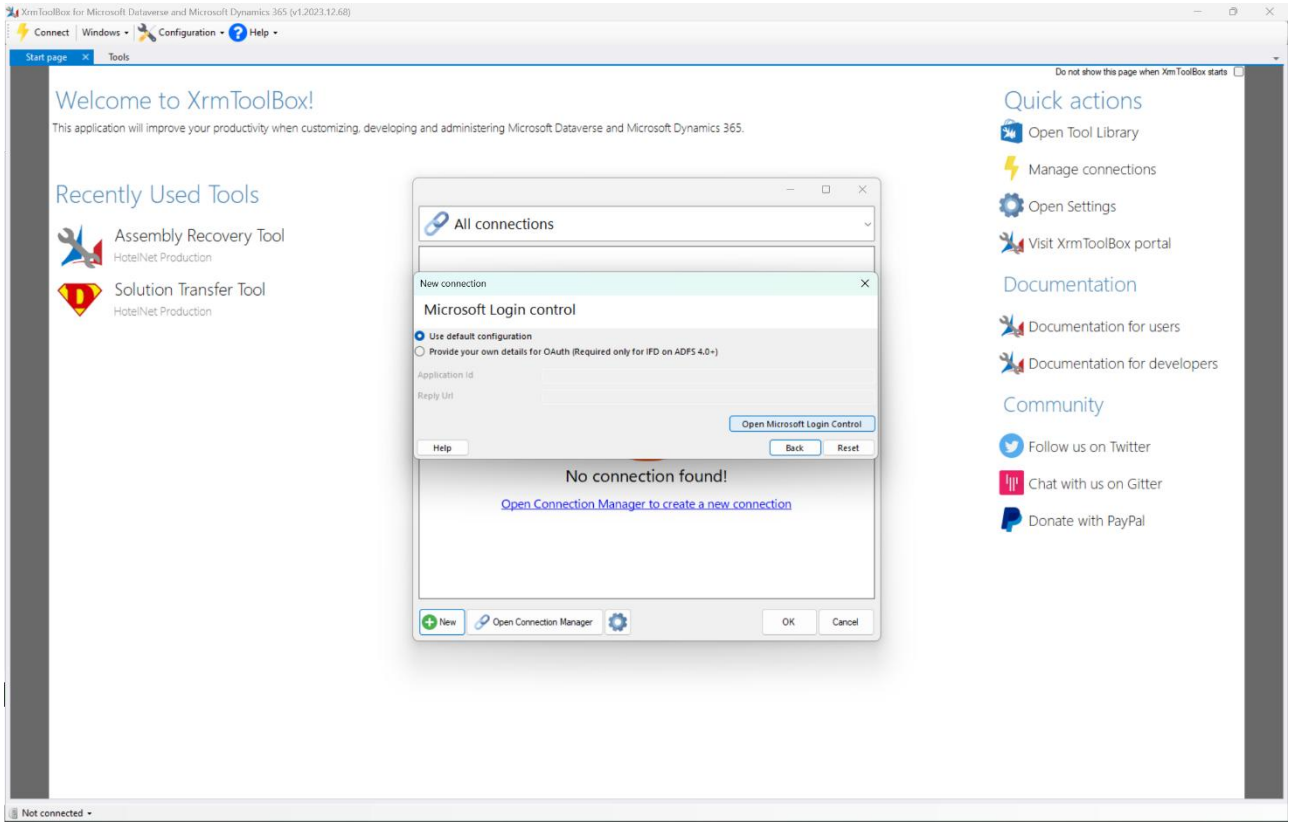
After installing, we look for the tool Assembly Recovery Tool. In this demo, we use it to recover the source code (which has been lost) of a plugin dll.

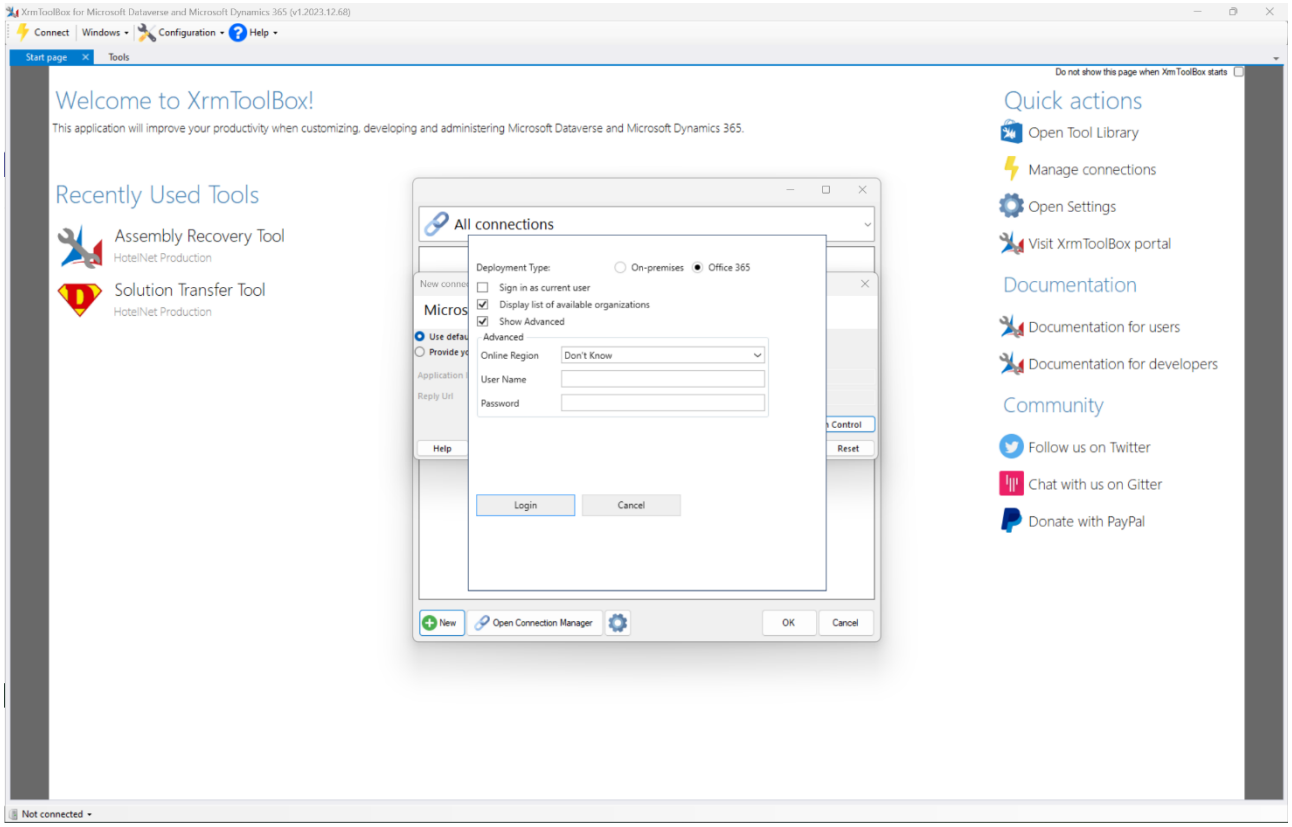


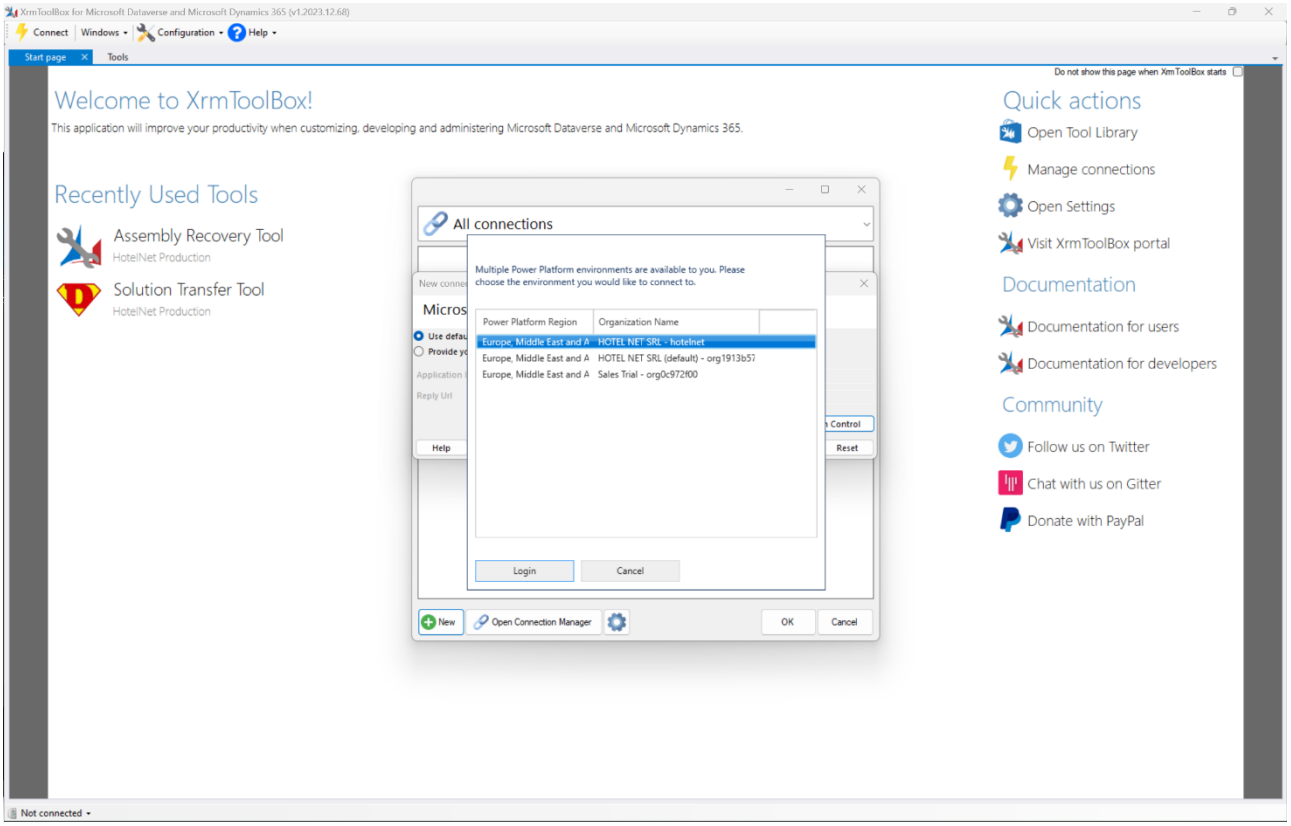
Click on + New to create a new connection to the target organization.

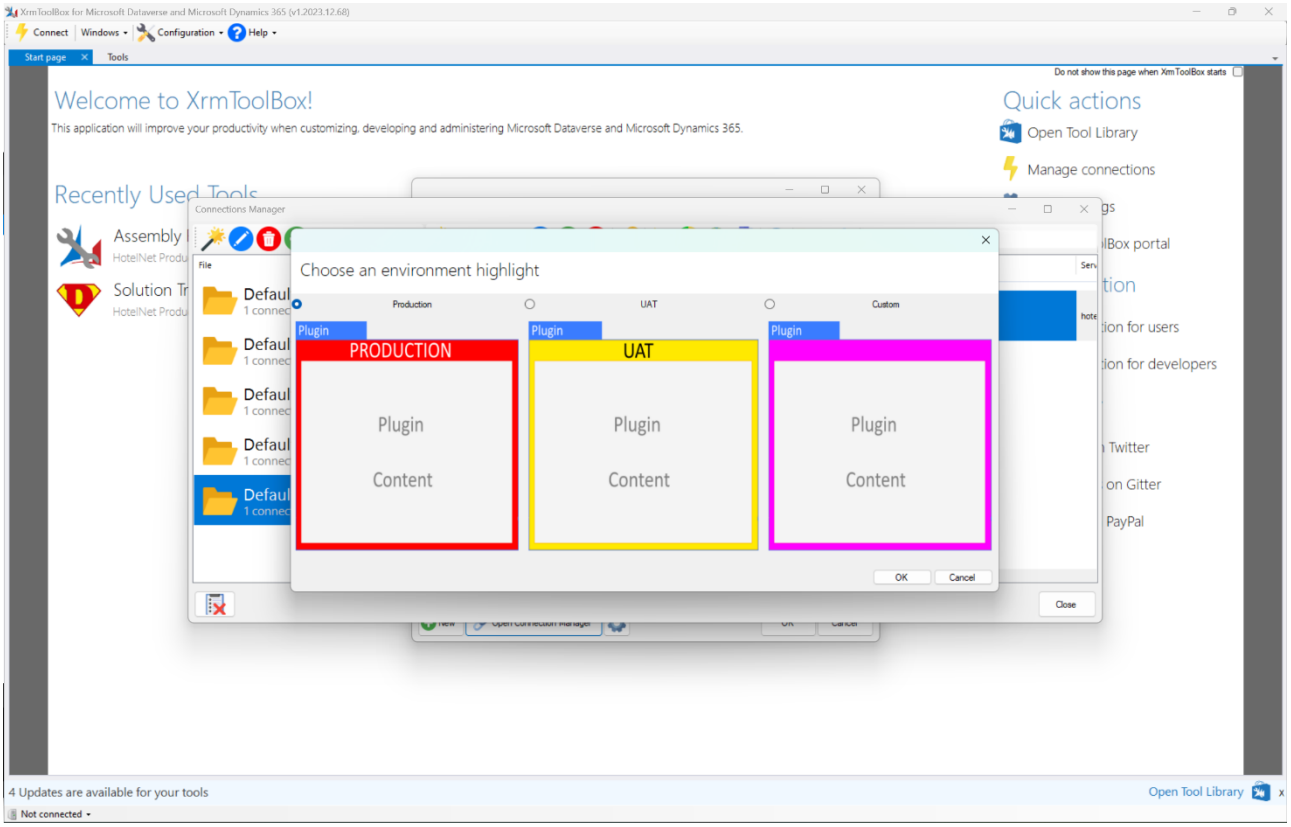




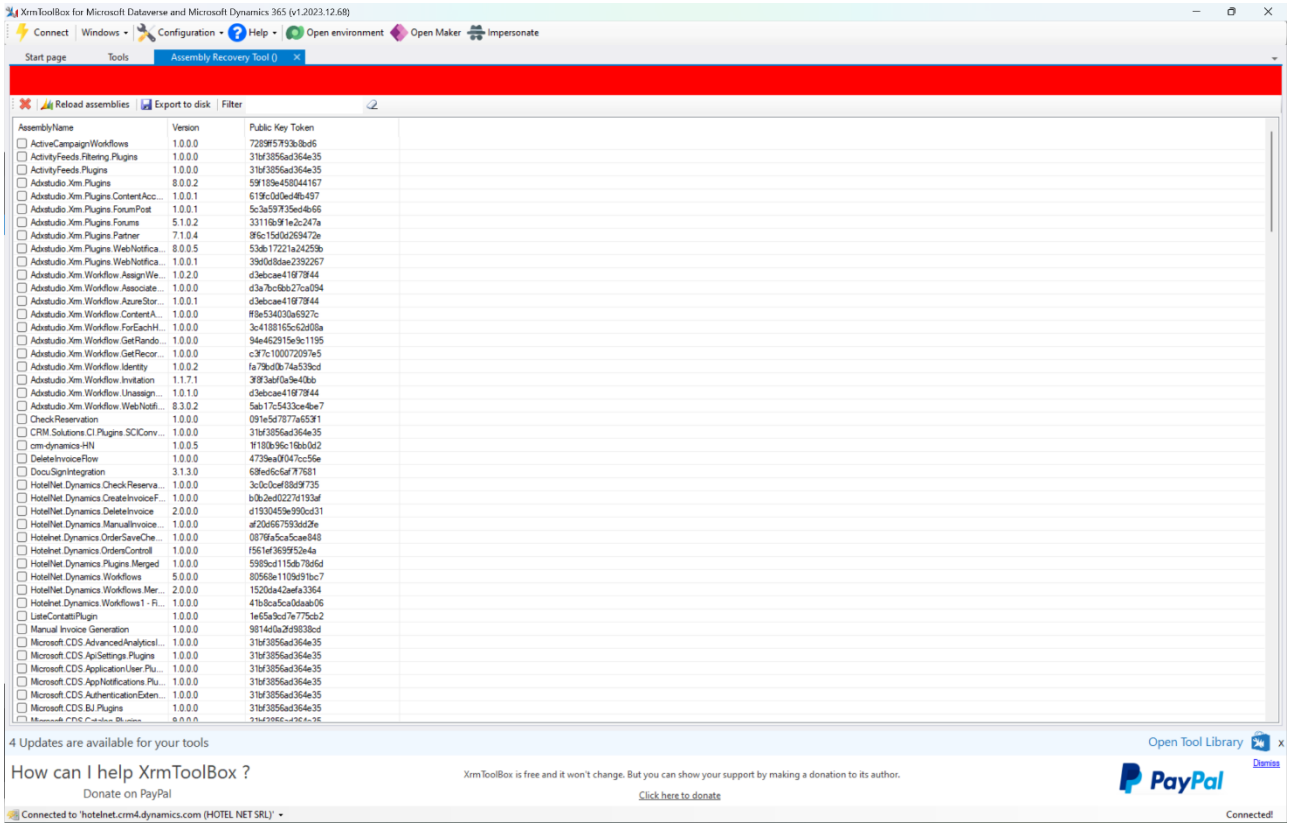

XrmToolBox for Microsoft Dataverse and Microsoft Dynamics 365 (v1.2023.12.68)
Connect
Windows
Configuration
Help
Start page
Tools
Do not show this page when XrmToolBox starts
Welcome to XrmToolBox!
This application will improve your productivity when customizing, developing and administering Microsoft Dataverse and Microsoft Dynamics 365.
Recently Used Tools
Assembly Recovery Tool
HotelNet Production
Solution Transfer Tool
HotelNet Production
All connections
New connection
Microsoft Login control
Use default configuration
Provide your own details for OAuth (Required only for IFD on ADFS 4.0+)
Application Id
Reply Url
Open Microsoft Login Control
Help
Back
Reset
No connection found!
Open Connection Manager to create a new connection
New
Open Connection Manager
OK
Cancel
Quick actions
Open Tool Library
Manage connections
Open Settings
Visit XrmToolBox portal
Documentation
Documentation for users
Documentation for developers
Community
Follow us on Twitter
Chat with us on Gitter
Donate with PayPal
Not connected


XrmToolBox for Microsoft Dataverse and Microsoft Dynamics 365 (v1.2023.12.68)
Connect
Windows
Configuration
Help
Start page
Tools
Do not show this page when XrmToolBox starts
Welcome to XrmToolBox!
This application will improve your productivity when customizing, developing and administering Microsoft Dataverse and Microsoft Dynamics 365.
Recently Used Tools
Assembly Recovery Tool
HotelNet Production
Solution Transfer Tool
HotelNet Production
All connections
Deployment Type:
On-premises
Office 365
Sign in as current user
Display list of available organizations
Show Advanced
Advanced
Online Region
Don't Know
User Name
Password
Login
Cancel
New
Open Connection Manager
OK
Cancel
Quick actions
Open Tool Library
Manage connections
Open Settings
Visit XrmToolBox portal
Documentation
Documentation for users
Documentation for developers
Community
Follow us on Twitter
Chat with us on Gitter
Donate with PayPal
Not connected


Welcome to XrmToolBox!
This application will improve your productivity when customizing, developing and administering Microsoft Dataverse and Microsoft Dynamics 365.
Recently Used Tools
Assembly Recovery Tool
HotelNet Production
Solution Transfer Tool
HotelNet Production
Quick actions
Open Tool Library
Manage connections
Open Settings
Visit XrmToolBox portal
Documentation
Documentation for users
Documentation for developers
Community
Follow us on Twitter
Chat with us on Gitter
Donate with PayPal
All connections
Multiple Power Platform environments are available to you. Please choose the environment you would like to connect to.
Power Platform Region
Organization Name
Europe, Middle East and A
HOTEL NET SRL - hotelnet
Europe, Middle East and A
HOTEL NET SRL (default) - org1913b57
Europe, Middle East and A
Sales Trial - org0c972f00
Login
Cancel
New
Open Connection Manager
OK
Cancel
Not connected

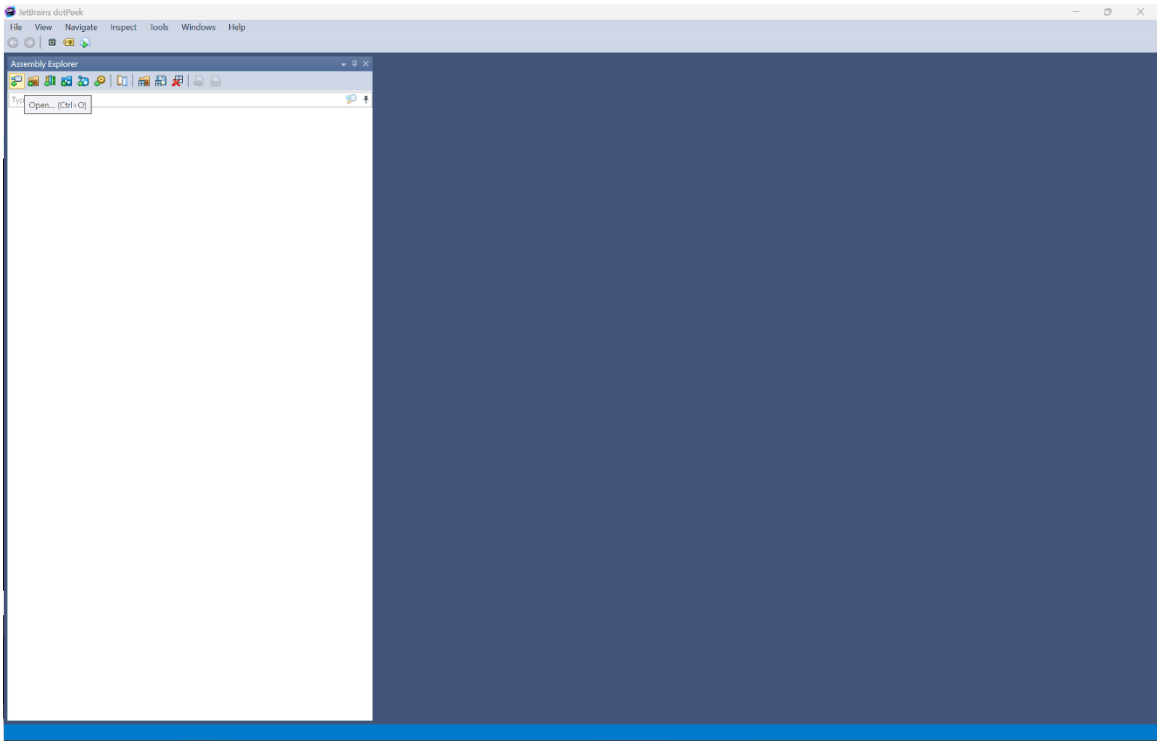

Welcome to XrmToolBox!
This application will improve your productivity when customizing, developing and administering Microsoft Dataverse and Microsoft Dynamics 365.
Quick actions
Open Tool Library
Manage connections
Connections Manager
Choose an environment highlight
Production
UAT
Custom
Plugin
PRODUCTION
UAT
Plugin
Content
OK
Cancel
Close
4 Updates are available for your tools
Open Tool Library
Not connected

Once connected, we can click open the Assembly Recovery Tool. Here we find all the dlls which have been registered.

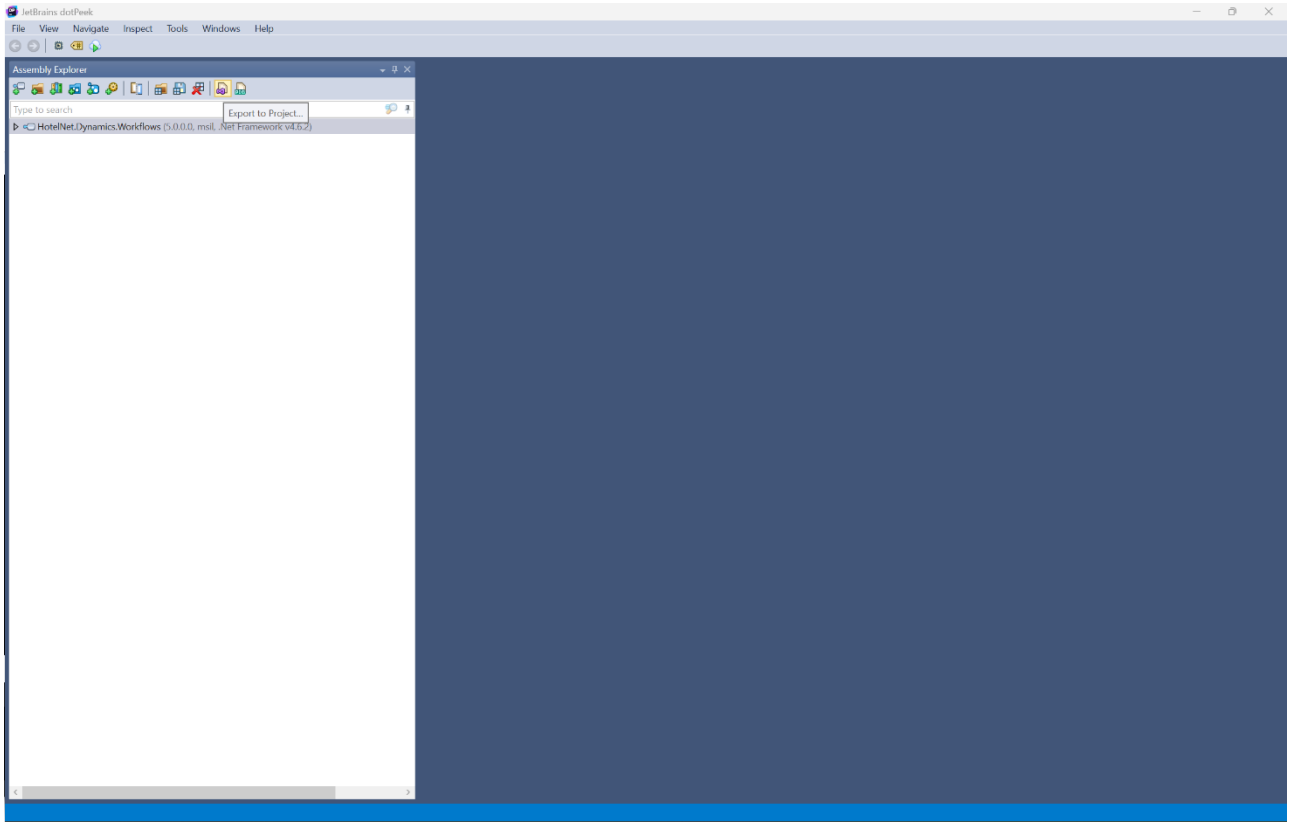


We can select one or more of these and click on Export to disk, which prompts us to select a location on our machine into which save the file.

Next, we need to download the dotPeek decompiler and assembly browser, which can be found at https://www.jetbrains.com/decompiler/download/#section=web-installer.



At this point, we open the file which have extracted using the previous tool and click on Export to Project.



This is operation will create a folder containing the project's files we were looking for.

Name	Date modified	Type	Size
.vs	1/24/2024 11:01 AM	File folder	
bin	1/24/2024 10:55 AM	File folder	
obj	1/24/2024 10:55 AM	File folder	
packages	1/24/2024 11:03 AM	File folder	
app	1/24/2024 11:03 AM	File di origine Config...	2 KB
AssemblyInfo.cs	1/24/2024 10:55 AM	C# Source File	1 KB
ChargeTheCustomer.cs	1/24/2024 11:18 AM	C# Source File	8 KB
HotelNet.Dynamics.Workflows_5.0.0.0.csproj	1/24/2024 11:03 AM	C# Project File	7 KB
HotelNet.Dynamics.Workflows_5.0.0.0.pdb	1/24/2024 10:55 AM	Program Debug Data...	16 KB
HotelNet.Dynamics.Workflows_5.0.0.0.sln	1/24/2024 10:55 AM	Visual Studio Solution	1 KB
packages	1/24/2024 11:03 AM	File di origine Config...	2 KB